EVM-Guard: Bytecode-Only Reentrancy Triage for Solidity 0.4–0.5

Devarsh Patel[1] and Chandru Vignesh C[1*]

[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, 632014, Tamil Nadu, India.

*Corresponding author(s). E-mail(s): chandru.vignesh@vit.ac.in
E-mail: devarsh.patel2024@vitstudent.ac.in

Abstract

Reentrancy remains one of the most frequently reported causes of loss in Ethereum contracts, and many deployed contracts are available only as bytecode. This paper describes the design and evaluation of **EVM-Guard**, a bytecode-only triage tool for reentrancy in contracts compiled with Solidity 0.4–0.5. Engine v5 builds a control-flow graph with an abstract-stack dataflow and flags an external call followed on some path by a storage write without an intervening mutex; calls that forward only the 2300-gas stipend are excluded, and each candidate is replayed in a `py-evm` sandbox. On SmartBugs-curated, v5 reached 93.5% reentrancy recall at 80.6% precision (95% CI 64.0–91.8); this figure is post hoc. When Slither was restricted to its `reentrancy-eth` and `reentrancy-no-eth` detectors, no significant difference in false positives was found ($p = 0.69$), whereas the bytecode analysers EVMLiSA and eThor produced significantly more false positives than v5 on every corpus on which they were run ($p \leq 1.3 \times 10^{-4}$). Of 3,214 deployed Solidity 0.4–0.5 contracts holding 45,217 ether, v5 flagged 522, with a median analysis time of 0.48 s per contract. A blind manual review of 120 flagged contracts identified one exploitable contract, a precision of 1.6% for v5 (95% CI 0.04–8.7); on the same contracts, v5 produced fewer false positives than strict Slither ($p = 0.021$). Most flags on deployed code concerned calls to fixed or trusted addresses, which suggests that the output of EVM-Guard on such code is best treated as a list for manual review.

Keywords: Smart Contract Security, Reentrancy, EVM Bytecode, Static Analysis, Path Feasibility, Z3, McNemar Test

1 Introduction

1.1 Motivation

Transactions on Ethereum cannot be reversed, and deployed bytecode cannot be patched in place. A contract that makes an external call before it has updated its own storage can therefore be re-entered and drained before the update takes effect. Surveys of attacks on deployed contracts have identified reentrancy, oracle manipulation and arithmetic errors as recurring sources of loss [1, 2], and transaction ordering can make such attacks easier to carry out [3]. Since the source code of many deployed contracts has not been published, there is a need for analysers that work on runtime bytecode alone.

1.2 Existing Analysis Approaches

1. **Static analysis over intermediate representations.** Source-level analysers such as Slither [4] translate the compiler's abstract syntax tree into an intermediate representation, so they cannot be applied when only bytecode is available. Bytecode analysers such as Securify [5] encode basic blocks as relational facts that are evaluated with Soufflé [6], but they abstract over stack depth and gas arithmetic, and evaluations of such tools have reported false-positive rates above 50% [7, 8].
2. **Symbolic execution.** Symbolic execution frameworks, including Mythril[1], Manticore [9] and Oyente [10], check path feasibility by passing path conditions to the Z3 solver [11]. On non-trivial contracts, loops that depend on storage and hashing instructions cause the number of paths to grow rapidly. In the evaluation reported here, Mythril timed out on 25.9% of the SmartBugs files and on 77.0% of the SolidiFI files. Exploit generators such as teEther [12] and fuzzers such as Echidna [13] and ConFuzzius [14] produce concrete inputs, although several of them require properties or a test harness to be written by hand.
3. **Property testing and deductive verification.** hevm [15] executes bytecode symbolically in order to check assertions written by the developer, and VerX [16] and KEVM [17] verify temporal properties against formal specifications. These tools give strong guarantees for the properties that are stated, but a specification has to be written for every contract.

EVM-Guard works on compiled runtime bytecode and needs neither annotations nor a test harness. Engine v5 evaluates fixed dataflow patterns over the recovered control-flow graph (CFG) and follows CFG reachability without solving path conditions. Engine v6 retains a v5 candidate only if Z3 cannot refute both a path to the candidate and a re-entrant run from the state at the call (Section 4.3). Neither engine proves that a hazard is absent (Section 4.1).

1.3 Research Questions

This study sought to answer the following research questions:

[1] https://github.com/Consensys/mythril; version 0.24.8 was used.

- **RQ1 (Detection precision):** Can CFG dataflow tracking within a function and the recognition of storage-slot mutexes reduce reentrancy false positives in comparison with source-level tools, while recall on benchmark suites remains high?
- **RQ2 (Replay):** To what extent can sandbox replay confirm static warnings without false alarms, and how much recall is lost when only confirmed findings are counted?
- **RQ3 (Generalisation):** How does precision on the benchmarks used during development compare with precision on held-out synthetic corpora, including one compiled with Solidity 0.8?
- **RQ4 (Deployed contracts):** How many deployed Solidity 0.4–0.5 contracts does EVM-Guard flag, at what cost, and how many of the flags prove exploitable on manual review?

1.4 Contributions

The main contributions of this paper are as follows.

1. **Hazard specifications and a reentrancy dataflow check.** Bit-vector specifications over $\mathcal{BV}_{256}$ are stated for six hazard classes, including read-only reentrancy (Section 3), together with the CFG dataflow checks and Z3 parameter templates that EVM-Guard implements (Table 3). The checks approximate these specifications and are not proved to decide them (Section 4.1). Calls restricted to the 2300-gas stipend are excluded from the reentrancy check.
2. **A bytecode analyser with sandbox replay.** EVM-Guard recovers basic-block CFGs from disassembled bytecode, computes an abstract-stack fixpoint, and replays candidate transactions in a py-evm sandbox, where a finding is confirmed only if a post-state criterion on balances or storage is met (Section 4).
3. **A path-feasibility engine.** Engine v6 adds return-site tracking, a Z3 path search under an unprivileged-attacker model that classifies each finding as same-function, cross-function or read-only reentrancy, and witness-seeded replay (Section 4.3). It was frozen before it was evaluated, and its results are reported alongside those of v5 on the same files (Section 7).
4. **Benchmark evaluation with paired tests.** Results are reported for SmartBugs-curated (143 files), SolidiFI (400 files), a labelled benchmark (389 files) and a series of pre-registered synthetic tests, against Slither, Mythril, EVMLiSA, eThor and the static stage of Sailfish, with exact McNemar tests on the same files. Figures obtained after the engine had been revised on a corpus are marked as post hoc (Sections 5 and 6).
5. **A study of deployed contracts.** Both engines were run on 3,214 deployed Solidity 0.4–0.5 contracts that hold ether, and a blind random sample of 120 flagged contracts was reviewed by hand (Section 8).

2 Related Work and Hazard Classes

Table 1 lists the tools that are closest to EVM-Guard, together with the input that each of them requires. Tool speeds are not included in the table, because the published

Table 1 Qualitative comparison of EVM analysis tools. "User spec." means the user must write assertions, harnesses or formal specifications before the tool can check a property.

Tool	Technique	Input	Requires	User spec.
Slither [4]	Pattern and dataflow rules on an IR	Source	Solidity source	No
Securify [5]	Datalog compliance patterns	Source or bytecode	solc for source; bytecode accepted	No
Mythril	Symbolic execution with SMT	Source or bytecode	Solidity or bytecode	No
Manticore [9]	Symbolic execution	Source or bytecode	Solidity or bytecode	Optional
Echidna [13]	Property-based fuzzing	Source	Properties to test	Yes
ConFuzzius [14]	Hybrid fuzzing with data dependencies	Source, or ABI of a deployed contract	Solidity source, or ABI and a chain node	No
VerX [16]	Deductive verification	Source	Formal specification	Yes
hevm [15]	Symbolic execution of assertions	Bytecode	Test harness	Yes
Gigahorse [18]	Decompilation to a register IR, Datalog clients	Bytecode	Bytecode only	No
eThor [19]	Sound static analysis by Horn-clause abstraction	Bytecode	Bytecode only	No
EVMLiSA [20]	Abstract interpretation (LiSA) with a reentrancy checker	Bytecode	Bytecode only	No
Sailfish [21]	Storage dependency graph with symbolic refinement	Source	Solidity source	No
EVM-Guard	CFG dataflow (v5); Z3 path feasibility and witness replay (v6)	Runtime bytecode	Bytecode only	No

figures were not obtained under a common protocol; timing is compared only for the runs reported in this paper (Section 5.11).

Bytecode reentrancy analysers.

eThor [19] abstracts EVM bytecode into Horn clauses and attempts to prove single-entrancy, a property that excludes reentrancy. Since the analysis is sound with respect to its semantics, a contract is reported as insecure whenever the property cannot be proved. EVMLiSA [20] constructs a sound CFG of the bytecode by abstract interpretation of the stack in the LiSA framework and applies a reentrancy checker to it. Gigahorse [18] decompiles bytecode into a register-based intermediate representation on which Datalog clients are run. Sailfish [21], in contrast, operates on source code: a storage dependency graph proposes reentrancy and transaction-ordering candidates, which a symbolic verifier then refines. EVM-Guard is compared with eThor, EVMLiSA and Sailfish in Section 6. Gigahorse is not used as a baseline, since it is a decompilation framework and has no reentrancy client of its own.

2.1 Positioning against hevm

hevm [15] checks assertions that a developer has written into a test harness, typically a Foundry property test, and explores the paths of the contract under their path conditions; a counterexample produced by hevm is therefore tied to the semantics of the contract. EVM-Guard, by contrast, needs no harness or assertion and applies a fixed set of checks to any runtime bytecode. The price of this generality is lower precision. Because the checks only approximate the specifications of Section 3, a finding indicates a hazard and is not a counterexample to a stated property. The two kinds of tool may therefore be regarded as complementary: EVM-Guard can be used to triage unannotated bytecode, and a property-testing tool can then confirm a suspected hazard once the property has been written.

2.2 Hazard Classes

EVM-Guard targets six hazard classes, which are mapped to SWC identifiers in Table 2. The specifications were written for this study; the citations refer to earlier work on the same class.

Table 2 Hazard classes targeted by EVM-Guard and the corresponding specifications (Sections 3.2 and 3.3).

Class	SWC	Hazard	Specification
A	SWC-107	A callee observes or changes state before the caller settles its own storage [22, 23].	Φ_{RO} (Def. 3)
B	SWC-101	Fixed-point division truncation that discards remainders.	Φ_{Prec} (Def. 6)
C	SWC-124	Proxy and implementation assigning one storage slot to variables of different widths.	Φ_{Slot} (Def. 5)
D	SWC-128	Loops over user-growable storage arrays that exceed a gas limit [24].	Φ_{DoS} (Def. 7)
E	Non-SWC	Lending logic that reads AMM spot reserves as a price oracle [1].	Φ_{Flash} (Def. 8)
F	SWC-112	DELEGATECALL target influenced by calldata.	Φ_{Taint} (Def. 9)

Class A: reentrancy (SWC-107).

A contract makes an external call before it has updated its own storage, and the callee re-enters it. In read-only reentrancy, a view function returns unsettled state to a third contract while the call is in progress. The check in EVM-Guard follows CFG edges from an external call to a later storage write in the same function and tests whether a mutex intervenes. Engine v5 does not distinguish between these variants, whereas engine v6 records whether the re-entrant run reaches the same call, a stale read in another function, or a stale read in a function that writes nothing (Definition 2).

Class B: division truncation (SWC-101).

Integer division $\lfloor x/y \rfloor$ discards the remainder, and division before multiplication increases the loss. SWC-101 also covers integer overflow and underflow, but EVM-Guard targets truncation only, a distinction that matters for the SmartBugs arithmetic category (Section 5.3).

Class C: storage-slot collision (SWC-124).

Upgradeable contracts execute implementation logic in the storage of the proxy through DELEGATECALL, and a mismatch between the two storage layouts can overwrite critical variables such as the owner.

Class D: loop gas exhaustion (SWC-128).

A loop over a dynamic storage array that any user can extend may grow until the call no longer fits within the gas limit.

Class E: spot-price oracle manipulation.

A flash loan can move the spot reserves of an automated market maker within a single transaction. The SWC registry has no identifier for this class (SWC-114 denotes transaction order dependence), and these findings are therefore referred to as Class E. The output files of the tool record them under the internal label SWC-114, which does

not refer to the registry entry. SmartBugs has no corresponding category, so these findings are reported but not scored.

Class F: delegatecall taint (SWC-112).

If calldata determines the target of a DELEGATECALL, an attacker can execute arbitrary code in the storage context of the caller.

3 Hazard Specifications over $\mathcal{BV}_{256}$

The definitions in this section are specifications; they do not describe what the implementation proves. Definitions 1–3 state the reentrancy property that EVM-Guard targets and the pattern that its check reports, and Section 4.1 and Table 3 describe how each engine approximates them. Definitions 5–9 specify the other hazard classes, for which results are reported but no claims are made.

3.1 Notation

An execution frame is $\mathcal{S} \triangleq \langle \sigma, \tau, \mu, \iota \rangle$, where $\sigma : \mathcal{BV}_{256} \to \mathcal{BV}_{256}$ is persistent storage, τ is EIP-1153 transient storage, μ is the machine state (stack, memory, program counter, gas), and ι is the transaction context, including the caller $\iota.caller$. Words are elements of $\mathcal{BV}_{256}$ with arithmetic modulo 2^{256}. For $v \neq 0$, $\mathrm{UDiv}(u, v) \triangleq \lfloor u/v \rfloor$ and $\mathrm{URem}(u, v) \triangleq u - v \cdot \mathrm{UDiv}(u, v)$, with $\mathrm{UDiv}(u, 0) = \mathrm{URem}(u, 0) = 0$, in agreement with the DIV and MOD opcodes in formal operational semantics of the EVM [17, 25]. The condition

$$\mathrm{NoMulOverflow}(u, v) \equiv (v = 0) \vee \big(\mathrm{UDiv}(u \cdot v, v) = u\big) \tag{1}$$

expresses that the wrapped product $u \cdot v$ equals the mathematical product. For a contract C, a public function f is the code reached from the dispatcher entry of one selector. A run $\sigma \xrightarrow{f, d, a} \sigma'$ executes f with calldata d and caller a from storage σ and ends in σ' without reverting. Inside a run, an external call site c is a CALL or CALLCODE instruction; σ_c is the storage when c executes, t_c the callee address and g_c the forwarded gas.

3.2 Reentrancy

Definition 1 (Unprivileged attacker) Let $K(C)$ be the 20-byte constants in the bytecode of C, and let $\mathrm{Fixed}(\sigma)$ be the addresses held, at any byte offset, in storage slots of σ whose index is a constant of the bytecode. A slot is *settable* if some public function writes it with a value derived from the caller, or from calldata on a path not guarded by a caller check. An address A is an unprivileged attacker for C in state σ if

$$\mathrm{Unpriv}(A, \sigma) \equiv A \notin K(C) \ \wedge \ A \notin \mathrm{Fixed}^{\neg\mathrm{set}}(\sigma), \tag{2}$$

where $\mathrm{Fixed}^{\neg\mathrm{set}}$ ranges over the slots that are not settable. The attacker sends the transaction, controls the code at A, and therefore controls what happens when A is called.

Definition 2 (Reentrancy hazard) Contract C has a reentrancy hazard at call site c of public function f if there are a state σ, calldata d and an address A with $\mathrm{Unpriv}(A, \sigma)$ such that a run $\sigma \xrightarrow{f,d,A}$ reaches c with

$$t_c = A \ \wedge \ g_c > 2300 \ \wedge \ \exists w \text{ after } c \text{ in } f : w \text{ writes slot } k, \tag{3}$$

and a re-entrant run $\sigma_c \xrightarrow{g,d',A}$ started from the storage at the call reaches either (i) c again, with $g = f$ (*same-function* reentrancy), or (ii) a read of slot k, with $g \neq f$ (*cross-function* reentrancy; *read-only* if g writes no storage and makes no call). The write w makes the re-entrant run observe a state that f has not settled.

Definition 2 is a single-transaction property relative to an arbitrary starting state σ. It requires neither that σ be reachable from deployment nor that re-entry be profitable. Section 4.1 records which parts of it each engine checks, and the labelling rubric of the study of deployed contracts (Section 8) adds the requirement of profit.

Definition 3 (Reentrancy pattern) Let $R(f)$ be the blocks reachable from the entry of f without entering another function's entry. Call site c in $R(f)$ matches the reentrancy pattern if the abstract gas bound of c exceeds 2300, some SSTORE w in $R(f)$ is reachable from c along CFG edges inside $R(f)$, and no mutex (a dominating write before c, a branch on the written slot before c, and a reset reachable after c) guards c.

Engine v5 reports every match of Definition 3. Engine v6 reports a match only if Z3 does not refute the two runs of Definition 2 under the attacker of Definition 1 (Section 4.3).

Remark 1 (What a rejection means) The symbolic model of engine v6 replaces every value that it cannot compute exactly (environment reads, results of external calls, hashes and unaligned memory) with an unconstrained symbol, so that every concrete run along a modelled path satisfies its path condition. If Z3 reports the path condition as unsatisfiable, no run can follow that path. This argument holds only for the modelled operations and for the bounded set of paths explored; it concerns the implementation and is not a machine-checked proof. A search that reaches one of its limits rejects nothing.

Definition 4 (Read-only reentrancy (Φ_{RO})) Let $\sigma_{\mathrm{transient}}$ be the storage state during an external call and $\sigma_{\mathrm{settled}}$ the state after the call. A contract violates read-only reentrancy safety if some public view function f_{view} returns different values in the two states:

$$\Phi_{\mathrm{RO}} \equiv \mathrm{InExternalCall}(\iota) \wedge \left(f_{\mathrm{view}}(\sigma_{\mathrm{transient}}) \neq f_{\mathrm{view}}(\sigma_{\mathrm{settled}}) \right) \tag{4}$$

This is the read-only case (ii) of Definition 2, stated in terms of the result of the view function.

3.3 Other hazard classes

Definition 5 (Proxy storage-slot collision (Φ_{Slot})) Let implementations C_1 and C_2 have slot assignments $\mathcal{L}_1, \mathcal{L}_2$ and types $\mathcal{T}_1, \mathcal{T}_2$. A collision exists if two variables mapped to the same

slot have different bit-widths:

$$\Phi_{\text{Slot}} \equiv \exists v_1 \in \text{Var}(C_1), v_2 \in \text{Var}(C_2) : \big(\mathcal{L}_1(v_1) = \mathcal{L}_2(v_2)\big) \wedge \big(\text{BitWidth}(\mathcal{T}_1(v_1)) \neq \text{BitWidth}(\mathcal{T}_2(v_2))\big) \tag{5}$$

Definition 6 (Fixed-point truncation (Φ_{Prec})) Let fixed-point division with scale 10^k ($k \in \{6, 18\}$) compute $\text{UDiv}(x \cdot 10^k, y)$. A truncation hazard exists if

$$\Phi_{\text{Prec}} \equiv \exists x, y \in \mathcal{BV}_{256} : \quad (y > 0) \wedge \text{NoMulOverflow}(x, 10^k) \wedge (\text{URem}(x \cdot 10^k, y) \neq 0) \wedge \Big(\text{UDiv}(\text{URem}(x \cdot 10^k, y) \cdot 10^k, y) > 0\Big) \tag{6}$$

This predicate holds for almost any division that leaves a non-zero remainder. It is therefore a necessary but not a sufficient condition for a truncation hazard, which is consistent with the high arithmetic false-positive rate reported in Section 5.3.

Definition 7 (Loop gas exhaustion (Φ_{DoS})) Out-of-gas and resource-exhaustion hazards have been studied extensively for the EVM [24, 26, 27]. Let N be the length of a dynamic storage array iterated by a loop, C_0 the base call cost and c_{step} the per-iteration cost, so $C_{\text{total}}(N) = C_0 + N \cdot c_{\text{step}}$. For a gas threshold G, a denial-of-service hazard exists if an unprivileged function can grow N without a static bound and

$$\Phi_{\text{DoS}}(G) \equiv \text{IsMutatorPublic}(f_{\text{mut}}) \wedge \neg\text{HasStaticBound}(N) \wedge \big(C_0 + N \cdot c_{\text{step}} > G\big) \tag{7}$$

The static Z3 template sets $G_{\text{block}} = 30 \times 10^6$, the gas limit of the genesis block of the sandbox, with $C_0 = 21{,}000$ and $c_{\text{step}} = 5{,}000$. A scaled threshold, $G_{\text{sandbox}} = 1.5 \times 10^6$, is used only by the replay oracle (Section 4.2), which limits each replayed transaction to G_{sandbox} gas and confirms a finding when the call exhausts that limit. A confirmation therefore shows unbounded growth at a reduced scale, not the exhaustion of a full block.

Definition 8 (Spot-price oracle manipulation (Φ_{Flash})) Let an AMM hold reserves x, y with fee $\phi \in [0, 1)$, and let the post-swap spot price be $P_1(\Delta x) = \frac{x + \Delta x(1-\phi)}{y - \Delta y}$. If a lending protocol prices collateral C and debt D at the spot price $P_0 = x/y$, a position solvent at P_0 can be pushed below the solvency threshold $H_{\min}$ when

$$\Phi_{\text{Flash}} \equiv \exists \Delta x : (0 < \Delta x \leq \Lambda_{\max}) \wedge \left(\frac{C \cdot P_0}{D} \geq H_{\min}\right) \wedge \left(\frac{C \cdot P_1(\Delta x)}{D} < H_{\min}\right) \tag{8}$$

Definition 9 (Delegatecall target taint (Φ_{Taint})) Let $T_{\text{target}} \in \mathcal{BV}_{160}$ be the DELEGATECALL target operand and AuthorizedTarget the set of permitted callees. On a path π with path condition $\text{PC}(\pi)$, the call is hijackable if

$$\Phi_{\text{Taint}} \equiv \exists \mathbf{d}, A_{\text{attacker}} : \text{PC}(\pi) \wedge (T_{\text{target}}(\mathbf{d}) = A_{\text{attacker}}) \wedge (A_{\text{attacker}} \notin \text{AuthorizedTarget}) \tag{9}$$

Remark 2 (Unimplemented specification) The interpreter of EVM-Guard does not model TSTORE or TLOAD, and it therefore has no check for divergence of transient-storage locks. Since no contract in the evaluation corpora uses EIP-1153, this omission does not affect the results.

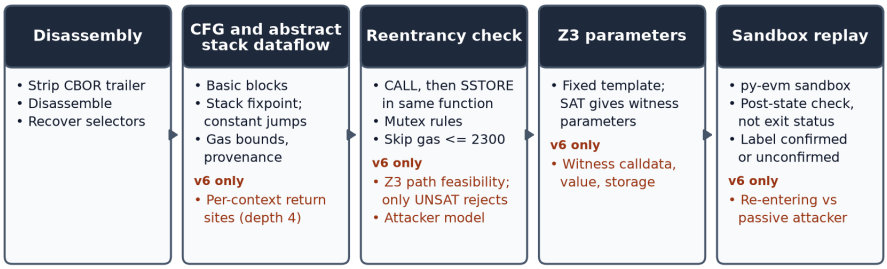


Fig. 1 The EVM-Guard pipeline. In engine v5, a CFG dataflow check decides whether a hazard is reported, a Z3 template supplies witness parameters, and the replay labels each finding confirmed or unconfirmed under a class-specific state criterion. Engine v6 adds, for reentrancy only, return-site tracking in the CFG fixpoint, a Z3 path-feasibility search that can reject a candidate only when the path condition is unsatisfiable, and a replay seeded with the Z3 witness.

4 System Architecture and Implementation

This section describes the architecture of EVM-Guard. The pipeline (Figure 1) consists of five steps, which are described below as four stages, since the Z3 step and the replay are implemented together.

1. **Disassembly.** The solc CBOR metadata trailer is removed, and the bytecode is disassembled with pyevmasm (src/symbolic_engine.py).
2. **CFG and abstract-stack dataflow** (src/evm_dataflow.py and src/evm_regions.py). Following the framework of abstract interpretation [28], the instructions are partitioned into basic blocks at each JUMPDEST and after each block terminator, and a worklist fixpoint is computed over an abstract value domain. Each stack slot holds a constant where one is known, together with provenance flags for calldata, the caller, storage (with the slot, if constant) and the DIV instructions that contributed to it. Jump edges are added only for constant JUMPDEST targets resolved during the fixpoint; dynamic jumps that cannot be resolved are counted and reported rather than guessed. Across the 216 runtime contracts of SmartBugs-curated, 180 JUMP blocks have no resolved successor, because the target is either unresolved or invalid (Section 10.2). The module also computes dominators by iterative bitmask intersection, the region reachable from each dispatcher selector without entering the entry of another selector, and the natural loops behind back edges.
3. **Hazard checks** (src/hazard_rules.py). One check per hazard class is applied to these structures (Table 3). A check that fires records its program points and the selectors whose regions contain them.
4. **Z3 templates and replay** (src/z3_encoder.py and src/replay_oracles.py). A check that fires instantiates a parameter template, which is solved with the Z3 SMT solver [11] and must be satisfiable. The replay oracle then constructs transactions and executes them in an isolated py-evm sandbox configured for the Cancun fork.

Algorithm 1 Triage pipeline of engine v5

Require: Runtime bytecode $\mathcal{B}$
Ensure: Findings $\mathcal{F}$, each labelled confirmed or unconfirmed

$\mathcal{B}' \leftarrow \textsc{StripMetadata}(\mathcal{B})$; $\mathcal{I} \leftarrow \textsc{Disassemble}(\mathcal{B}')$
$\mathcal{G} \leftarrow \textsc{AnalyzeFlow}(\mathcal{I})$ ▷ CFG and abstract-stack fixpoint
$\mathcal{D} \leftarrow \textsc{Dominators}(\mathcal{G})$; $\mathcal{R} \leftarrow \textsc{FunctionRegions}(\mathcal{G})$
$\mathcal{F} \leftarrow \emptyset$
for each hazard class h **do**
    $e \leftarrow \textsc{Check}(h, \mathcal{G}, \mathcal{D}, \mathcal{R})$ ▷ evidence or $\perp$
    if $e \neq \perp$ **then**
        $(r, \mathcal{M}) \leftarrow \textsc{SolveTemplate}(h)$
        if r = SAT **then**
            $ok \leftarrow \textsc{ReplayOracle}(h, \mathcal{B}, \mathcal{M}, e)$
            $\mathcal{F} \leftarrow \mathcal{F} \cup \{(h, e, ok)\}$
        end if
    end if
end for
return $\mathcal{F}$ ▷ an empty $\mathcal{F}$ is not a proof of safety

Dispatcher selectors are recovered syntactically. A PUSH4 value that is compared by EQ within five instructions of a JUMPI whose target is a JUMPDEST is taken as a selector, and the target as the entry of its function. This recovery is a pattern match and is not part of the dataflow analysis; it finds at least one selector in 206 of the 216 runtime contracts.

4.1 What Each Check Implements

Table 3 lists, for each hazard class, the check that decides whether a finding is reported and the Z3 template that supplies its parameters. Whether a template is satisfiable depends only on its constants and on the flags set by the check. In engine v5, Z3 therefore supplies witness parameters for a fixed template and does not decide whether the contract satisfies the specification of Section 3. An empty result does not establish that a contract is safe, and a finding does not establish that it is exploitable.

Scope of the guarantee (engine v5).

The analysis consists of a fixed sequence of passes over the CFG of one runtime bytecode, followed by Z3 queries with a 5 s timeout. It always terminates and is deterministic apart from Z3 timeouts. The checks follow CFG edges, use dominators and resolve natural loops, but they do not solve path conditions, and dynamic jump targets that cannot be resolved to a constant are abandoned. A finding thus shows only that the bytecode contains the dataflow pattern of Table 3 and that the template is satisfiable with the reported parameters. Engine v6 extends this guarantee for reentrancy (Section 4.3).

Table 3 Checks implemented in EVM-Guard. The check, evaluated over the CFG, dominators, function regions and loops, decides whether a finding is reported; the Z3 template supplies witness parameters.

Class	Check (CFG dataflow)	Z3 template
SWC-107	A CALL or CALLCODE whose gas operand is not a constant at most 2300 (the stipend), in a function region, from which an SSTORE in the same region is reachable along CFG edges, unless a mutex guards it. The mutex is a dominating write of 1 or 2, a storage-derived branch before the call, and a reset to 0 or 1 reachable after it.	Balances and supplies in $\mathcal{BV}_{256}$ whose transient and settled values differ.
SWC-101	A DIV or SDIV result that reaches a later MUL operand or an SSTORE value along the abstract-stack dataflow.	64-bit projected a, b with $b > 1000$ and $\mathrm{URem}(1000a, b) > 0$; the candidate is scaled by 10^{14} and its remainder recomputed in 256 bits.
SWC-112	A DELEGATECALL whose target is not a constant, derives from calldata, and is not dominated by a block containing CALLER.	SAT if and only if the taint flag is set.
SWC-124	A DELEGATECALL in a contract that contains the EIP-1967 implementation-slot constant, or whose target derives from a storage read.	Collision query over a fixed layout pair (slot 0: address vs uint256). Layouts are not recovered from the bytecode.
SWC-128	A natural loop whose header lies in a dispatcher function region, is not dominated by a CALLER block, and has an exit branch that tests a storage read. Exits to revert blocks are ignored.	$\Phi_{\mathrm{DoS}}(G_{\mathrm{block}})$ with IsMutatorPublic assumed true.
Class E	DIV present, at least two SLOAD, and no SSTORE, CALL, DELEGATECALL or TIMESTAMP. This is a presence heuristic, not a dataflow check.	Fixed reserves, with a borrow that shifts the spot price by at least 25%.

4.2 Sandbox Replay

The replay oracle installs the runtime bytecode as account code in a py-evm chain, so no constructor is executed. Each replay starts from a fresh chain, and all transactions are sent from a single externally owned account. The criteria implemented for each class are as follows.

- **SWC-107:** an attacker contract forwards calls to the victim and, when it receives value without calldata, re-enters the victim once with the stored calldata. Deposit and withdrawal calldata are built from the recovered selectors (at most 8 of each). A finding is confirmed only if the victim calls back into the attacker and the attacker's gain in the withdrawal transaction exceeds its deposit.
- **SWC-112:** a probe contract writes 0x2a to slot 0x5afe. The finding is confirmed only if a call to a selector with the probe address as argument sets that slot of the victim's own storage to 0x2a.
- **SWC-124:** slot 0 is initialised with an address-width value, and the finding is confirmed only if a call overwrites it with a value of at least 160 bits.
- **SWC-128:** the slot that bounds the loop is set to 10^6; findings whose exit test has no constant slot are not replayed. The finding is confirmed only if a call with this storage exhausts the G_{sandbox} limit (an out-of-gas halt) while the same call on the original storage succeeds.
- **SWC-101 and Class E:** no oracle exists, because neither a truncation nor a spot-price effect leaves a post-state that the sandbox could check without an economic model of the contract. These findings are never confirmed.

A transaction that merely completes without reverting is never counted as a confirmation. Total findings (F) and confirmed findings (C) are reported separately (Table 6),

and Table 7 gives the scores for the four scored classes when only confirmed findings are counted.

4.3 Engine v6: Return Sites, Path Feasibility and Witness Replay

Engine v6 consists of v5 and four additional components, each controlled by one switch in src/engine_options.py; the checks for the other hazard classes are unchanged. The sources were frozen before the evaluation and are kept byte for byte in archive/engine_v6/ (Section 7).

Return-site tracking.

Engine v5 merges the return addresses of internal helpers. A helper that is called from two functions therefore links the continuations of both callers, and the checked-arithmetic helpers emitted by Solidity 0.8 cut the dataflow at the dispatcher (Section 10.3). In v6, the fixpoint analyses each block once per context, a context being the sequence of JUMPDEST addresses on the abstract stack (at most 4), with no more than 32 contexts per block (src/evm_dataflow.py). A return can then jump only to the address pushed in its own context.

Path feasibility.

For each candidate that matches Definition 3 and passes the mutex rules, src/-path_feasibility.py searches the CFG depth first with a symbolic executor over $\mathcal{BV}_{256}$ (src/symbolic_path.py). In this model, calldata is an array of 256-bit words, and SHA3 is an uninterpreted function whose results are constrained to be at least 2^{128}, so that hashed slots cannot alias constant slots. Memory is a map from words to words, and after an external call every slot k becomes $\mathrm{If}(t_c = A, \mathrm{fresh}, \sigma(k))$, which means that only a call to the attacker can change storage. The search addresses the two parts of Definition 2. First, it looks for a path from the entry of f through c to the write w with $t_c = A$ and $g_c > 2300$. Second, it looks for a re-entrant run from σ_c that reaches c again (same-function) or reads a slot updated by w in another public function (cross-function or read-only). The attacker constraint of Definition 1 excludes the PUSH20 constants of the bytecode and the addresses held in fixed slots that no unprivileged caller can set. Each solver call is limited to 1 s, each search to 4 s and 3,000 blocks, and each contract to 30 s. A candidate is rejected only if the solver reports unsatisfiability (Remark 1); if a search reaches a limit, the candidate is kept and recorded as *unknown*.

Witness replay.

A second Z3 query fixes the roles in the sandbox (attacker, owner, and a victim balance of at most 2.5 ether) and yields the calldata, the call value and the starting storage, with actual keccak values for mapping slots (src/replay_witness.py). The transaction is then replayed from that storage twice, once with a re-entering attacker and once with a passive receiver, and the finding is confirmed in this tier only if re-entry yields strictly

more. Results of this tier are reported separately from those of the default-state replay of v5 (Section 4.2), which is unchanged.

5 Empirical Evaluation on SmartBugs-Curated

An earlier version of this work evaluated EVM-Guard on a corpus of 226 contracts that had been built and labelled by the first author. That evaluation is no longer reported, because its labels were not independent and its contracts were derived from a small number of templates; this section uses an external benchmark instead.

5.1 Corpus

The evaluation uses the SmartBugs-curated dataset [7, 29] at commit 230e6491 (2024-06-16). The dataset contains 143 Solidity files, each of which is labelled with one category in the dataset's vulnerabilities.json: unchecked low-level calls (52), reentrancy (31), access control (18), arithmetic (15), bad randomness (8), denial of service (6), time manipulation (5), front running (4), other (3) and short addresses (1). Each file was compiled with the solc version listed in the dataset's versions.csv (0.4.0–0.5.0). All 143 files compile, yielding 216 runtime contracts, and their length ranges from 6 to 1,254 source lines (mean 63.4). Since no file is labelled as safe, a file is treated as a negative for category X if it carries no label in X.

5.2 Evaluation Protocol and Baselines

The SmartBugs protocol (docs/EXTERNAL_EVAL_PROTOCOL.md) was frozen before the first engine run, and its SHA-256 digest, which covers lines 1–39 of the file, is recorded in docs/EXTERNAL_EVAL_FREEZE.sha256. Addendum A (lines 40–58 of the same file) fixes the mapping from the detector names of each baseline tool to the SmartBugs categories; it was written after the first EVM-Guard run but before any baseline was run on this corpus. Only that first run, with engine v1, was pre-registered. It reached 71.0% reentrancy recall (22 of 31, CI [52.0, 85.8]) at 44.9% precision. The engine was subsequently revised on this corpus: v2 is the first revision, v3 adds the gas-stipend bound (Section 5.8), and v5, the final engine, adds the PUSH0 decoder and the slot-based mutex rule (Section 5.10). All SmartBugs figures for v2, v3 and v5 are therefore post hoc.

Category mapping.

SWC-107 maps to reentrancy, SWC-101 to arithmetic, SWC-128 to denial of service, and SWC-112 and SWC-124 to access control. Class E has no SmartBugs category and is not scored. Unchecked low-level calls, bad randomness, front running, time manipulation, short addresses and the category *other* lie outside the scope of the tool, and no recall is claimed for them.

Baselines.

Slither 0.11.6 was run on each source file with the solc version listed in the dataset and with –json output, without a detector filter, and its findings were mapped to categories by detector name. Reentrancy uses reentrancy-eth, -no-eth, -benign, -events and -unlimited-gas (the mapping of Addendum A, used in Table 4). Slither itself describes the last three detectors as reporting benign, event-ordering and gas-forwarding patterns rather than exploitable reentrancy. Section 6 therefore also reports a strict mapping that counts only reentrancy-eth and reentrancy-no-eth. Denial of service uses calls-loop and costly-loop. Access control uses arbitrary-send-eth, arbitrary-send-erc20, suicidal, controlled-delegatecall, unprotected-upgrade, tx-origin, protected-vars and incorrect-modifier. Since Slither has no overflow detector, none of its findings maps to arithmetic. Mythril 0.24.8 was run on each source file with the solc version listed in the dataset, with -o json –execution-timeout 50, and the process was terminated after 60 s. Its SWC identifiers were mapped as follows: SWC-107 to reentrancy, SWC-101 to arithmetic, SWC-113 and SWC-128 to denial of service, and SWC-105, 106, 112, 115 and 124 to access control.

Scoring.

Scoring is carried out at the level of files. A file is flagged for category X if any of its runtime contracts yields a finding that maps to X, and all findings reported by the engine are counted, whether or not they are confirmed by replay. For each category, TP, FP, FN and TN are reported over all 143 files, with exact 95% Clopper–Pearson intervals [30] for recall, precision and FPR. Two tools are compared on the same files with an exact McNemar test [31], that is, a two-sided binomial test on the files flagged by exactly one of the two tools, applied separately to the negatives (false positives) and to the positives (true positives). This paired test was preferred to a comparison of intervals, since overlapping marginal intervals do not show that a paired difference is absent. Baseline errors and timeouts are counted as unflagged and reported separately. For EVM-Guard, latency is the wall time of the analysis call for each runtime contract, summed over the file and excluding replay.

Platform.

All runs were carried out on one machine (Apple M3, eight cores, 8 GB of memory, macOS) with Python 3.12, z3-solver 5.1.0.0 and py-evm 0.12.1b1, under the time limits given for each tool.

5.3 Detection Results

Table 4 and Figure 2 present the results for every scored category and all three tools. Differences between tools were tested with the paired McNemar test described in Section 5.2.

Table 4 Per-category detection on SmartBugs-curated (143 files, 216 runtime contracts). EVM-Guard rows are the final engine v5, post hoc. Recall, precision and FPR reported with exact 95% Clopper–Pearson confidence intervals. A tool with no mapped detector for a category flags nothing in it.

Category	Tool	TP	FP	FN	TN	Recall (%)	Precision (%)	FPR (%)
Reentrancy	**EVM-Guard**	**29**	**7**	**2**	**105**	**93.5 [78.6, 99.2]**	**80.6 [64.0, 91.8]**	**6.2 [2.5, 12.5]**
	Slither	29	31	2	81	93.5 [78.6, 99.2]	48.3 [35.2, 61.6]	27.7 [19.6, 36.9]
	Mythril	7	30	24	82	22.6 [9.6, 41.1]	18.9 [8.0, 35.2]	26.8 [18.9, 36.0]
Denial of service	**EVM-Guard**	**5**	**34**	**1**	**103**	**83.3 [35.9, 99.6]**	**12.8 [4.3, 27.4]**	**24.8 [17.8, 32.9]**
	Slither	2	14	4	123	33.3 [4.3, 77.7]	12.5 [1.6, 38.3]	10.2 [5.7, 16.6]
	Mythril	1	24	5	113	16.7 [0.4, 64.1]	4.0 [0.1, 20.4]	17.5 [11.6, 24.9]
Access control	**EVM-Guard**	**2**	**0**	**16**	**125**	**11.1 [1.4, 34.7]**	**100.0 [15.8, 100.0]**	**0.0 [0.0, 2.9]**
	Slither	8	37	10	88	44.4 [21.5, 69.2]	17.8 [8.0, 32.1]	29.6 [21.8, 38.4]
	Mythril	13	11	5	114	72.2 [46.5, 90.3]	54.2 [32.8, 74.4]	8.8 [4.5, 15.2]
Arithmetic	**EVM-Guard**	**1**	**80**	**14**	**48**	**6.7 [0.2, 31.9]**	**1.2 [0.0, 6.7]**	**62.5 [53.5, 70.9]**
	Slither	0	0	15	128	0.0 [0.0, 21.8]	n/a	0.0 [0.0, 2.8]
	Mythril	0	0	15	128	0.0 [0.0, 21.8]	n/a	0.0 [0.0, 2.8]

Table 5 Run outcomes and per-file wall time on SmartBugs-curated. Baseline times include `solc` compilation; EVM-Guard times cover analysis of compiled bytecode only, excluding replay.

Tool	Completed	Errors	Timeouts	Median (ms)	Mean ± SD (ms)
EVM-Guard	**143**	**0**	**0**	**87**	**128 ± 170**
Slither	141	2	0	289	304 ± 135
Mythril	86	20	37	26,025	30,288 ± 24,942

Reentrancy.

The figures in Table 4 refer to the final engine v5 and are post hoc. EVM-Guard v5 reached 93.5% recall (29 of 31, CI [78.6, 99.2]) at 80.6% precision, whereas engine v2, before the stipend correction, had the same recall at 53.7% precision (Section 5.8). Slither also reached 93.5% recall, with the same 29 true positives, but its precision was 48.3%; Mythril reached 22.6% recall (7 of 31, CI [9.6, 41.1]). EVM-Guard v5 produced 7 false positives (FPR 6.2%) and Slither 31 (FPR 27.7%). Of the 112 negatives, 24 were flagged by Slither alone and none by v5 alone (McNemar $p = 1.2 \times 10^{-7}$); against Mythril, the corresponding counts were 4 and 27 ($p = 3.4 \times 10^{-5}$). This difference from Slither, however, disappears under the strict mapping (Section 6). Most of the 25 false positives of engine v2 arose from calls that forward only the stipend, which v3 and v5 exclude.

Reentrancy errors of v5.

Each of the remaining errors was examined individually. All seven false positives are files labelled as unchecked low-level calls, and Slither flags all seven as well. One of them is a genuine reentrancy that the label omits: `withdraw()` in `keepMyEther` sends the caller's balance with `call.value` and sets it to zero only afterwards. Three can be

Table 6 Engine v5 findings on 216 runtime contracts by hazard class (F) and the number confirmed by default-state replay (C). SWC-101 and Class E have no replay oracle. File-level precision of the confirmed findings is given in Table 7.

Class	Static Findings (F)	Replay Confirmed (C)
SWC-107 (reentrancy)	41	4
SWC-128 (loop gas)	43	3
SWC-112 (delegatecall)	1	1
SWC-124 (slot collision)	1	0
SWC-101 (truncation)	106	n/a
Class E (spot price)	2	n/a
Total	194	8

Table 7 EVM-Guard v5 file-level scores counting only replay-confirmed findings (95% Clopper–Pearson interval for recall).

Category	TP	FP	FN	TN	Recall (%)	Precision (%)
Reentrancy	4	0	27	112	12.9 [3.6, 29.8]	100.0
Denial of service	2	1	4	136	33.3 [4.3, 77.7]	66.7
Access control	1	0	17	125	5.6 [0.1, 27.3]	100.0
Arithmetic	0	0	15	128	0.0 [0.0, 21.8]	n/a

Table 8 Evaluation on the SolidiFI bug-injection benchmark (400 files: 350 with injected bugs, 50 clean). The protocol was frozen before the held-out engine v2 run, whose results are given in the text; the EVM-Guard rows are the final engine v5, post hoc. Recall, precision and FPR with exact 95% Clopper–Pearson intervals. Denial of service has no injected class in this corpus and is not scored.

Category	Tool	TP	FP	FN	TN	Recall (%)	Precision (%)	FPR (%)
Reentrancy	**EVM-Guard**	**50**	**35**	**0**	**315**	**100.0 [92.9, 100.0]**	**58.8 [47.6, 69.4]**	**10.0 [7.1, 13.6]**
	Slither	50	150	0	200	100.0 [92.9, 100.0]	25.0 [19.2, 31.6]	42.9 [37.6, 48.2]
	Mythril	2	7	48	343	4.0 [0.5, 13.7]	22.2 [2.8, 60.0]	2.0 [0.8, 4.1]
Arithmetic	**EVM-Guard**	**45**	**315**	**5**	**35**	**90.0 [78.2, 96.7]**	**12.5 [9.3, 16.4]**	**90.0 [86.4, 92.9]**
	Slither	0	0	50	350	0.0 [0.0, 7.1]	n/a	0.0 [0.0, 1.0]
	Mythril	0	0	50	350	0.0 [0.0, 7.1]	n/a	0.0 [0.0, 1.0]

reached only by a privileged caller, since sendTokenAw in daoPOLSKA and WithdrawToHolder in TokenBank are restricted to the owner and deliver in TownCrier may be called only by its SGX address. In the remaining three, the callee is fixed by the owner: donate in two copies of PoCGame pays an address set by the owner, and CryptoPanda calls contracts configured by its CEO. Since engine v5 has no attacker model, it reports all six of these cases, whereas engine v6 removes the three privileged-caller

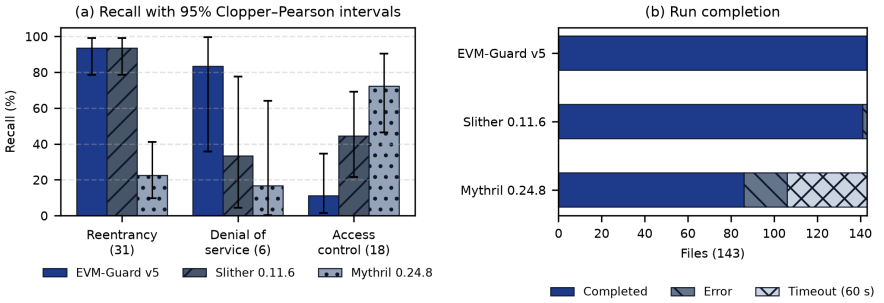


Fig. 2 Results on SmartBugs-curated (143 files). (a) Recall of EVM-Guard v5, Slither 0.11.6 and Mythril 0.24.8 for reentrancy, denial of service and access control, with exact 95% Clopper–Pearson intervals; arithmetic is reported in Table 4. (b) Run completion: Mythril timed out on 37 files (25.9%) and failed on 20, whereas EVM-Guard completed all 143.

cases and both copies of PoCGame (Section 7). Of the two false negatives, reentrancy_bonus is missed because its write lies in an internal function that is reached through a shared return address, which v5 merges, and spank_chain_payment is missed because its vulnerable contract contains an unlinked library placeholder and is not analysed (Section 5.11).

Denial of service.

EVM-Guard flagged 5 of the 6 labelled files (83.3% recall, CI [35.9, 99.6]), Slither 2 (33.3%, CI [4.3, 77.7]) and Mythril 1 (16.7%, CI [0.4, 64.1]). With only six positives, these differences in recall cannot be regarded as established. EVM-Guard produced 34 false positives (12.8% precision, FPR 24.8%), against an FPR of 10.2% for Slither and 17.5% for Mythril. The loop check requires only an exit test derived from storage within a public function region, which may explain its false positives.

Access control.

Mythril achieved the highest recall in this category, 72.2% (13 of 18, CI [46.5, 90.3]), at 54.2% precision, followed by Slither with 44.4% (8 of 18, CI [21.5, 69.2]) at 17.8% precision. EVM-Guard flagged 2 of the 18 files (11.1%, CI [1.4, 34.7]) and produced no false positive. Several access-control files rely on patterns that EVM-Guard does not target, such as authorisation through tx.origin (2 of the 18 labelled files) and unprotected self-destructs. Its delegatecall and proxy checks produced no false positive on this corpus, but they cover only direct call-hijacking hazards.

Arithmetic (construct mismatch).

SmartBugs uses the arithmetic category for integer overflow and underflow, whereas the SWC-101 check of EVM-Guard targets division truncation. This row therefore shows how the truncation check behaves on code labelled for overflow; it does not measure the detection of truncation. EVM-Guard flagged 1 of the 15 labelled files (6.7%, CI [0.2, 31.9]) and 80 other files, giving 1.2% precision and a false-positive rate

of 62.5%. This high rate is consistent with the predicate of Definition 6, which holds for almost any division that leaves a remainder. Neither Slither nor Mythril reported an overflow on this corpus (0% recall, CI [0.0, 21.8]). Slither has no overflow detector, so the protocol maps none of its findings to arithmetic; Mythril completed 12 of the 15 arithmetic files (with 2 errors and 1 timeout), yet reported no SWC-101 finding on any of the 143 files.

Summary.

In summary, engine v5 reached 80.6% reentrancy precision on this corpus (post hoc) and 100% precision on access control, although the latter rests on only two flagged files. Precision was considerably lower for denial of service (12.8%) and arithmetic (1.2%), which is consistent with the conservative abstractions used for loops and truncation. Reentrancy is the only category in which the engine matched the recall of Slither. Outside reentrancy, v2, v3 and v5 flag the same files.

5.4 Replay-Confirmed Findings

Across the 216 runtime contracts, engine v5 produced 194 static findings, 86 of which belong to the four classes that have a replay oracle (Table 6). The oracles confirmed 8 of these findings: 4 for reentrancy, 3 for loop gas and 1 for delegatecall. All four confirmed reentrancy findings and the confirmed access-control finding are true positives with respect to the file labels, whereas the three confirmed denial-of-service findings include one false alarm (66.7% precision). Overall, 7 of the 8 confirmed findings are true positives, a confirmation precision of 87.5%. Recall on the confirmed findings alone is low: 12.9% for reentrancy, 33.3% for denial of service and 5.6% for access control (Table 7). This is to be expected, since the oracles confirm only those hazards whose exploit can be reconstructed from the recovered selectors with generic attacker or probe contracts; a finding whose exploit requires specific arguments remains unconfirmed even when the static finding is correct. Replay therefore acts as a filter of high precision rather than as a means of finding hazards. With respect to the second research question, the oracle does not confirm warnings without false alarms, since one of the 8 confirmed findings is a false alarm, and recall falls to between 5.6% and 33.3% when only confirmed findings are counted.

5.5 Case Studies from the Corpus

The two cases below are SmartBugs-curated contracts that EVM-Guard flags and the replay oracle confirms; line numbers refer to the sources in the dataset.

Reentrancy: simple_dao.sol ***(labelled reentrancy).***

SimpleDAO keeps a credit mapping. Its withdraw(uint) function checks the balance (line 17), calls msg.sender.call.value(amount)() (line 19) and only then decrements credit[msg.sender] (line 20), so the external call precedes the storage write. EVM-Guard recovers the dispatcher selectors, finds a CFG path from the CALL block to the SSTORE block within the region of withdraw, finds no mutex on that path,

and reports SWC-107. During replay, the attacker first calls $\mathrm{donate(attacker)}$ with 1 ether, which credits its address, and then calls $\mathrm{withdraw}$ for 1 ether. The transfer from the victim re-enters the attacker, which calls $\mathrm{withdraw}$ again before the balance has been decremented. The attacker ends the transaction with 2 ether on a deposit of 1 ether, and the oracle therefore confirms the finding.

Loop gas: $\mathrm{dos_address.sol}$ ***(labelled denial of service).***

DosGas keeps a dynamic array, $\mathrm{creditorAddresses}$, at storage slot 0. The public function $\mathrm{addCreditors}$ pushes 350 entries per call, and the public function $\mathrm{emptyCreditors}$ checks $\mathrm{creditorAddresses.length} > 1500$ and then clears the array. EVM-Guard reports a loop in the region of $\mathrm{emptyCreditors}$ (selector $\mathrm{0x0d870b7f}$) whose exit test reads storage slot 0, the length of the array, and whose function is not guarded by a CALLER check. During replay, the oracle sets slot 0 to 10^6, after which a call to $\mathrm{emptyCreditors}$ exhausts the limit of $G_{\text{sandbox}} = 1.5 \times 10^6$ gas, whereas the same call on the original storage completes. This confirms unbounded growth at the reduced scale of the sandbox. The result is consistent with $\Phi_{\text{DoS}}(G_{\text{block}})$, but it does not show that a full block of 30 million gas would be exhausted.

5.6 Evaluation on the SolidiFI Bug-Injection Benchmark

In order to assess generalisation to an independent corpus, EVM-Guard was evaluated on the SolidiFI benchmark [8], which injects bugs of seven types into 50 base contracts (50 files per type) and also provides the 50 clean base contracts. The protocol ($\mathrm{docs/HELDOUT_EVAL_PROTOCOL.md}$, with its SHA-256 digest in $\mathrm{docs/HELDOUT_EVAL_FREEZE.sha256}$) fixed the corpus commits, the label mappings and the compiler settings before the analysis. Only reentrancy and arithmetic are scored, since EVM-Guard has no detector for the other injected types and the corpus contains no denial-of-service or delegatecall class. All 400 files compiled with solc 0.5.12, yielding 984 runtime contracts, and the baselines were run with the same settings as on SmartBugs.

The held-out run used engine v2, which flagged all 50 reentrancy files (100%, CI [92.9, 100]) together with 130 false positives, a precision of 27.8% and a false-positive rate of 37.1%. Table 8 reports the final engine v5, whose figures are post hoc: it produced 35 false positives, giving 58.8% precision and a false-positive rate of 10.0%, the same as v3. Slither flagged all 50 positives with 150 false positives (25.0% precision, FPR 42.9%). Mythril finished only 84 of the 400 files (308 timeouts and 8 errors) and flagged 2 of the 50 reentrancy files (4% recall, 22.2% precision, FPR 2.0%). Since every SolidiFI file is derived from one of the same 50 base contracts, some false positives probably reflect reentrancy patterns that are already present in the unlabelled base code. On the 50 clean contracts alone, v2 flagged 6 for reentrancy (12%) and v5 flagged 5 (10%).

The arithmetic row again reflects the mismatch between overflow labels and a truncation check. EVM-Guard flagged 45 of the 50 files with injected overflows (90%), but it also flagged 45 of the 50 clean contracts, because of division truncation in the

shared base code. Of the 350 files without an arithmetic label, 315 were flagged (FPR 90.0%, precision 12.5%). Neither Slither nor Mythril reported an arithmetic finding.

Replay confirmed no reentrancy finding on this corpus, although it confirmed 16 SWC-128 findings, which are not scored here. In the file that was examined (Reentrancy/buggy_1.sol), the injected withdrawal functions pay out from per-sender balance mappings that no function of the contract credits; in a fresh sandbox, the attacker therefore has no balance to withdraw, and the gain criterion cannot be met. The same contract also exposes 29 selectors, more than the limit of 8 candidates used by the oracle. The absence of confirmations on this corpus therefore reflects the limited reach of the oracle rather than an absence of the injected bugs.

Since the engine was changed after this run (v3 and v5), the SolidiFI figures for those versions are post hoc, and a fair test of an improvement in precision requires a corpus that has not yet been used.

5.7 Evaluation on a Labelled Benchmark Suite

SolidiFI has no denial-of-service, tx.origin or SELFDESTRUCT class, and its files share 50 base contracts. EVM-Guard was therefore also evaluated on an open repository of labelled contracts[2] at commit 8b903aa, whose 389 Solidity files are labelled by folder: reentrancy (81), arithmetic (65), unsafe delegatecall (12), tx.origin (10), unsafe SELFDESTRUCT (11), and five further classes (unchecked send, TOD, time manipulation, bad randomness, gasless send), plus 17 safe contracts. The evaluation protocol (docs/HELDOUT2_EVAL_PROTOCOL.md, with its SHA-256 digest in docs/HELDOUT2_EVAL_FREEZE.sha256) fixed the mappings between labels and categories and the compiler configurations, and the catalogue records the commit and the SHA-256 digest of each source.

The folder labels were mapped to the SmartBugs categories as follows. Reentrancy corresponds to SWC-107 and arithmetic to SWC-101, with the same mismatch between overflow and truncation as before. Access control groups the delegatecall folder (SWC-112), the tx.origin folder (SWC-115) and the SELFDESTRUCT folder (SWC-106), as in the SmartBugs mapping; since EVM-Guard has no check for tx.origin or SELFDESTRUCT, those 21 files can only count as misses. The five remaining classes are negatives for every scored category. Denial of service is not evaluated, because no file carries that label. Thirteen files do not compile with the available compilers; they remain in the scoring as unflagged.

The held-out run again used engine v2, which reached 97.5% reentrancy recall (79 of 81) at 46.2% precision (CI [38.6, 54.0]) and a false-positive rate of 29.9%. The final engine v5 (Table 9, post hoc) has the same recall at 76.0% precision (CI [66.6, 83.8]) and a false-positive rate of 8.1%. Slither matched this recall at 41.6% precision and a false-positive rate of 36.0%. Mythril completed 143 of the 389 files (231 timeouts and 15 errors) and reached 17.3% recall; a rerun with a limit of 300 s completed only 2 of the 231 files that had timed out, which raised its access-control recall from 33.3% to 39.4% and left reentrancy unchanged. Of the 92 reentrancy false

[2] https://github.com/bit-smartcontract-analysis/smartcontract-benchmark

Table 9 Results on the labelled benchmark (389 files, of which 376 compile; the other 13 count as unflagged). EVM-Guard rows refer to the final engine v5 and are post hoc; the results of the held-out engine v2 are given in the text. Recall, precision and FPR with exact 95% Clopper–Pearson intervals. Denial of service is not evaluated, because the benchmark has no such label.

Category	Tool	TP	FP	Recall [CI]	Precision [CI]	FPR [CI]
Reentrancy (81 pos.)	**EVM-Guard**	79	25	97.5 [91.4, 99.7]	76.0 [66.6, 83.8]	8.1 [5.3, 11.7]
	Slither	79	111	97.5 [91.4, 99.7]	41.6 [34.5, 48.9]	36.0 [30.7, 41.7]
	Mythril	14	32	17.3 [9.8, 27.3]	30.4 [17.7, 45.8]	10.4 [7.2, 14.3]
Arithmetic (65 pos.)	**EVM-Guard**	46	245	70.8 [58.2, 81.4]	15.8 [11.8, 20.5]	75.6 [70.6, 80.2]
	Slither	0	0	0.0 [0.0, 5.5]	n/a	0.0 [0.0, 1.1]
	Mythril	0	0	0.0 [0.0, 5.5]	n/a	0.0 [0.0, 1.1]
Access control (33 pos.)	**EVM-Guard**	7	0	21.2 [9.0, 38.9]	100.0 [59.0, 100.0]	0.0 [0.0, 1.0]
	Slither	22	152	66.7 [48.2, 82.0]	12.6 [8.1, 18.5]	42.7 [37.5, 48.0]
	Mythril	11	25	33.3 [18.0, 51.8]	30.6 [16.3, 48.1]	7.0 [4.6, 10.2]

positives of v2, 58 lie in the TOD folder, compared with 5 of the 25 of v5. These files were scored as negatives on the basis of their label, without manual review.

For access control, EVM-Guard v5 flagged 7 files, all in the delegatecall folder, without a false positive (v2 had flagged 6). Its recall over the 33 positives is 21.2%, since the tx.origin and SELFDESTRUCT files lie outside its checks; Slither reached 66.7% recall at 12.6% precision, and Mythril 33.3% recall at 30.6% precision. In the arithmetic category, where overflow labels are again compared with a truncation check, EVM-Guard v5 flagged 46 of the 65 overflow files (70.8% recall) and 245 of the 324 negatives (FPR 75.6%, precision 15.8%), whereas neither Slither nor Mythril reported an arithmetic finding. Replay confirmed 4 reentrancy findings and 2 access-control findings, all of which are true positives.

On SolidiFI and on this benchmark, the reentrancy precision of v5 is thus 58.8% and 76.0%; both figures are post hoc.

5.8 Ablation: Effect of the Gas-Stipend Bound

Solidity compiles address.transfer and address.send into a CALL whose gas operand is computed as $2300 \cdot \mathrm{ISZERO}(v)$ rather than given as a constant. Such calls forward at most the stipend of 2300 gas, which is not enough for the callee to modify storage. Before the Istanbul fork, SSTORE required at least 5,000 gas, and since EIP-2200 and EIP-2929 an explicit check halts execution with an out-of-gas error whenever the remaining gas is at most 2300. A call that forwards only the stipend therefore cannot re-enter the caller and modify its storage.

The effect of this exclusion was assessed by comparing EVM-Guard with and without the stipend bound. In the full engine, abstract stack values carry sound upper bounds, which are propagated through ISZERO (bound 1), MUL (product of the bounds) and AND (the smaller bound), and any call whose bound does not exceed 2300 gas is excluded.

Table 10 compares reentrancy detection on the three real benchmarks for engine v2, the stipend-corrected engine v3 and the final engine v5. The stipend bound raised

Table 10 Reentrancy (SWC-107) precision and FPR (%) for engine v2 (no stipend bound), v3 (stipend bound) and v5 (final engine) on the three real benchmarks. All figures are post hoc. Recall is the same for the three versions on each benchmark.

Corpus (positives)	**v2**		**v3**		**v5**	
	Prec.	FPR	Prec.	FPR	Prec.	FPR
SmartBugs-curated (31)	53.7	22.3	78.4	7.1	**80.6**	**6.2**
SolidiFI (50)	27.8	37.1	58.8	10.0	**58.8**	**10.0**
Labelled benchmark (81)	46.2	29.9	75.2	8.4	**76.0**	**8.1**

precision from 53.7% to 78.4% on SmartBugs-curated, from 27.8% to 58.8% on SolidiFI and from 46.2% to 75.2% on the labelled benchmark, while recall remained unchanged. The slot-based mutex rule of v5 removed one further false positive on SmartBugs-curated (80.6%) and one on the labelled benchmark (76.0%). All of these figures are post hoc. Of the 58 files in the TOD folder of the labelled benchmark that v2 flags for reentrancy, the stipend bound removes 53, and in each of these 53 sources value is sent only through transfer or send.

In addition, the components of the final engine were disabled one at a time on SmartBugs-curated (results/ablation/). This ablation covers reentrancy only, and recall was unchanged in every configuration. Removing the stipend exclusion, or applying it only to constant gas operands, raised the number of false positives from 7 to 22 (56.9% precision). Removing the slot-based mutex rule raised it to 8 (78.4%), and removing both mutex rules gave the same result. Disabling the PUSH0 decoder, the removal of the CBOR trailer, or the dominance requirement of the mutex rule did not change any reentrancy flag. A configuration that reproduces engine v2 gave 25 false positives (53.7%), in agreement with the archived v2 run for every file.

5.9 First Pre-Registered Synthetic Test (Engine v3)

This test applied engine v3 to a corpus that had not been used at any stage of development. The protocol (docs/INJECTED05_EVAL_PROTOCOL.md), together with its generator, catalogue and engine manifest, was frozen before the run, and the digests are recorded in docs/INJECTED05_EVAL_FREEZE.sha256. The corpus consists of 210 contracts compiled with solc 0.5.12, in seven groups of 30: positives with an injected reentrancy or denial-of-service pattern, and five groups of negatives (checks before effects, a mutex-guarded call, a payout through transfer, a bounded loop, and no injected pattern). Since the labels are assigned by construction and the families are few, the test measures the detector on its own injection patterns and provides no evidence about real contracts; its purpose was to measure precision on a fresh sample.

The results are shown in Table 11. For reentrancy, EVM-Guard flagged all 30 injected contracts, and all of its 30 false positives belong to the mutex-guarded group; the checks-before-effects, transfer and bounded-loop groups produced none. Slither

Table 11 First pre-registered synthetic test (210 Solidity 0.5.12 contracts, engine v3). Recall, precision and FPR with exact 95% Clopper–Pearson intervals. Each category is scored on its own positives and negatives.

Category (positives)	Tool	TP	FP	Recall [CI]	Precision [CI]	FPR [CI]
Reentrancy (30)	EVM-Guard	30	30	100.0 [88.4, 100.0]	50.0 [36.8, 63.2]	16.7 [11.5, 22.9]
	Slither	30	60	100.0 [88.4, 100.0]	33.3 [23.7, 44.1]	33.3 [26.5, 40.7]
	Mythril	30	57	100.0 [88.4, 100.0]	34.5 [24.6, 45.4]	31.7 [24.9, 39.0]
Denial of service (30)	EVM-Guard	30	0	100.0 [88.4, 100.0]	100.0 [88.4, 100.0]	0.0 [0.0, 2.0]
	Slither	0	0	0.0 [0.0, 11.6]	n/a	0.0 [0.0, 2.0]
	Mythril	0	0	0.0 [0.0, 11.6]	n/a	0.0 [0.0, 2.0]

and Mythril also flagged all 30 positives, with 60 and 57 false positives respectively. For denial of service, EVM-Guard flagged all 30 injected contracts without a false positive, whereas Slither and Mythril flagged none of them. Mythril timed out on 3 of the 210 files, which count as unflagged.

The cause of the mutex failure could be identified. In the mutex-guarded contracts, locked is written through a masked read-modify-write, so the stored value is not the constant that the mutex rule expects and the guard is not recognised. The engine was not changed on the basis of this corpus, since the corpus would then have been spent; the corrected mutex rule was instead tested on a new pre-registered corpus (Section 5.10).

5.10 Final Pre-Registered Synthetic Test (Engine v5)

The final engine, v5, consists of v3 with the $\mathrm{PUSH0}$ decoder and the slot-based mutex rule. It contains no return-site fallback, because that fallback lowered precision on the real benchmarks (SmartBugs reentrancy from 78.4% to 55.6%, SolidiFI from 58.8% to 23.6%). Under Protocol 6 ($\mathrm{docs/FINAL_V5_EVAL_PROTOCOL.md}$, frozen before the run, with digests in $\mathrm{docs/FINAL_V5_EVAL_FREEZE.sha256}$), this engine was run once on a new corpus of 210 contracts compiled with solc 0.5.12. Before that, Protocol 5 ($\mathrm{docs/FRESH_V4_EVAL_PROTOCOL.md}$) had tested an intermediate engine, v4, which included the return-site fallback, on two further corpora of 210 contracts, $\mathrm{injected05b}$ (solc 0.5.12) and $\mathrm{injected8b}$ (solc 0.8.20). Table 13 reports that run as it was frozen. Engine v4 was not adopted, because its fallback lowered precision on the real benchmarks, and both of its corpora are now spent. The corpus of Protocol 6 has the same seven groups as before and had not been used for any engine version. As its labels are assigned by construction, the result again concerns the injection patterns of the detector and not precision on real contracts. Table 12 reports the results.

On Solidity 0.5.12, engine v4 obtained the same result as v5 later did. On Solidity 0.8.20, its fallback, which links every return of a shared helper to every caller, allowed it to reach the call sites, but it also flagged all 60 mutex-guarded and checks-before-effects negatives and found none of the denial-of-service contracts. Engine v6 replaces this fallback with return sites tracked per context (Section 4.3).

Table 12 Final pre-registered synthetic test (Protocol 6; 210 Solidity 0.5.12 contracts). Recall, precision and FPR with exact 95% Clopper–Pearson intervals. Mythril had 12 timeouts and 1 error, which count as unflagged.

Category (positives)	Tool	TP	FP	Recall [CI]	Precision [CI]	FPR [CI]
Reentrancy (30)	EVM-Guard v5	30	0	100.0 [88.4, 100.0]	100.0 [88.4, 100.0]	0.0 [0.0, 2.0]
	Slither	30	60	100.0 [88.4, 100.0]	33.3 [23.7, 44.1]	33.3 [26.5, 40.7]
	Mythril	30	47	100.0 [88.4, 100.0]	39.0 [28.0, 50.8]	26.1 [19.9, 33.2]
Denial of service (30)	EVM-Guard v5	30	0	100.0 [88.4, 100.0]	100.0 [88.4, 100.0]	0.0 [0.0, 2.0]
	Slither	0	0	0.0 [0.0, 11.6]	n/a	0.0 [0.0, 2.0]
	Mythril	0	0	0.0 [0.0, 11.6]	n/a	0.0 [0.0, 2.0]

Table 13 Protocol 5: intermediate engine v4 (return-site fallback) on two pre-registered synthetic corpora of 210 contracts (30 reentrancy, 30 denial of service, 150 negatives). Precision with exact 95% Clopper–Pearson intervals; recall is 100% wherever TP is 30. Engine v4 was not adopted.

		Reentrancy			Denial of service		
Corpus	**Tool**	TP	FP	Precision [CI]	TP	FP	Precision [CI]
injected05b	EVM-Guard v4	30	0	100.0 [88.4, 100.0]	30	0	100.0 [88.4, 100.0]
(solc 0.5.12)	Slither	30	60	33.3 [23.7, 44.1]	0	0	n/a
	Mythril	30	54	35.7 [25.6, 46.9]	0	0	n/a
injected8b	EVM-Guard v4	30	60	33.3 [23.7, 44.1]	0	0	n/a
(solc 0.8.20)	Slither	30	60	33.3 [23.7, 44.1]	0	0	n/a
	Mythril	30	60	33.3 [23.7, 44.1]	0	0	n/a

The denial-of-service result shows that the baselines do not flag this template; it does not show that EVM-Guard detects denial of service in real contracts. On the earlier synthetic corpus (Section 5.9), where v3 had reached 50% precision because of the mutex defect, v5 also scores 100%, but that figure is post hoc, since the corpus had already been used.

5.11 Runtime and Completion

Table 5 reports the run outcomes and the wall time per file for the analysis stage, excluding replay. EVM-Guard completed all 143 files without a timeout or an error, with a median analysis time of 87 ms (mean 128 ± 170 ms). Slither completed 141 files and failed on 2, with a median of 289 ms (mean 304 ± 135 ms), and Mythril completed 86 files, with 37 timeouts (25.9%) and 20 crashes and a median of 26,025 ms (mean 30,288 ± 24,942 ms). It should be noted that the times of the baselines include compilation from source, whereas EVM-Guard analyses compiled bytecode. One of the 216 runtime contracts, LedgerChannel in spank_chain_payment.sol, contains an unlinked library placeholder and is not analysed, so its file is scored on its other contracts. When the placeholder is linked, the file is also flagged for reentrancy, and reentrancy recall rises to 96.8% (30 of 31, CI [83.3, 99.9]) at 81.1% precision. Since unfinished files count as unflagged, the timeouts and errors of Mythril lower its recall in Table 4.

6 Comparison with Bytecode Analysers and a Strict Slither Mapping

Setup.

All of the tools discussed in this section were run on the same files as EVM-Guard, and the run files are available in results/baselines_round3/. Slither 0.11.6 was run again with its detector names recorded, so that the same run could be scored under the mapping of Addendum A and under the strict mapping (Section 5.2). EVMLiSA (commit 44ae6dd, built with JDK 21) received the same runtime bytecode as EVM-Guard, with the solc metadata trailer removed, since EVMLiSA otherwise stops on the truncated PUSH instruction to which the trailer decodes; it was given 60 s per runtime contract (src/eval/run_evmlisa.py). As the download links of the eThor authors were no longer available, the SmartBugs image smartbugs/ethor:2023 was used with the SmartBugs command line. This image is built for amd64 only and therefore ran under emulation on the arm64 evaluation machine, which is why it was given 120 s per contract. At a mean of 25 s per file under emulation, and with no more than three containers in memory at a time, eThor was run on SmartBugs-curated and on the Protocol 6 synthetic corpus; within the time available, it was not run on SolidiFI and the labelled benchmark (a further 1,764 runtime contracts). Sailfish analyses source code, and its symbolic verifier is a Racket program. In the authors' image (holmessherlock/sailfish), the Racket runtime aborted on the evaluation machine under both QEMU and Rosetta emulation, even for the command racket -e "(display 1)", so the complete tool could not be run. Its static stage, which proposes the candidates that the verifier would refine, was run instead. Since the verifier can only remove candidates, the recall of the static stage is an upper bound on the recall of the complete tool. In all cases, a file is flagged if the tool reports at least one reentrancy finding on any of its contracts.

Results.

The comparison is presented in Table 14. Under the strict mapping, the false positives of Slither fall from 31 to 9 on SmartBugs-curated, from 150 to 42 on SolidiFI and from 111 to 29 on the labelled benchmark, without any loss of recall. Against strict Slither, v5 has significantly fewer false positives only on SolidiFI (7 negatives flagged by strict Slither alone and none by v5 alone, $p = 0.016$) and on the synthetic corpus (30 negatives flagged by strict Slither alone and none by v5 alone, $p = 1.9 \times 10^{-9}$). On SmartBugs-curated (2 negatives flagged by v5 alone and 4 by Slither alone, $p = 0.69$) and on the labelled benchmark (5 and 9, $p = 0.42$), the difference is not significant, and on SolidiFI v6 has significantly more false positives than strict Slither (14 and none, $p = 1.2 \times 10^{-4}$). The comparisons with mapped Slither in Section 5.3 therefore depend on the mapping, and the comparison with strict Slither is the more appropriate one. EVMLiSA and eThor flag almost every positive, but both produce significantly more false positives than v5 on every corpus on which they were run. eThor attempts to prove single-entrancy, a stronger property than exploitable reentrancy, and reports a contract as insecure whenever the proof fails, which is likely to account for its false

Table 14 Reentrancy against Slither under two mappings and three further analysers, on the same files. Mapped Slither counts every `reentrancy-*` detector; strict Slither counts `reentrancy-eth` and `reentrancy-no-eth`. EVMLiSA had 60 s and eThor 120 s per runtime contract; files a tool did not finish count as unflagged, and the last column gives how many. The McNemar column compares false positives with EVM-Guard v5 (and v6 where it was run): files flagged only by EVM-Guard / only by the tool, with the two-sided p-value. Sailfish's static stage is reported only on SmartBugs-curated, because it crashed on most reentrancy-positive files of the other corpora (text). eThor was run on two corpora only (text).

Corpus (pos./files)	Tool	TP	FP	Recall	Precision [CI]	FP vs v5 / vs v6 (p)	Unfinished
SmartBugs-curated (31/143)	EVM-Guard v5	29	7	93.5	80.6 [64.0, 91.8]	–	0
	Slither, mapped	29	31	93.5	48.3 [35.2, 61.6]	0/24 (1.2×10^{-7})	2
	Slither, strict	29	9	93.5	76.3 [59.8, 88.6]	2/4 (0.69)	2
	EVMLiSA	30	25	96.8	54.5 [40.6, 68.0]	1/19 (4×10^{-5})	4
	eThor	30	60	96.8	33.3 [23.7, 44.1]	5/58 (1.7×10^{-12})	27
	Sailfish, static stage	24	4	77.4	85.7 [67.3, 96.0]	4/1 (0.38)	5
SolidiFI (50/400)	EVM-Guard v5	50	35	100.0	58.8 [47.6, 69.4]	–	0
	EVM-Guard v6	50	56	100.0	47.2 [37.4, 57.1]	–	0
	Slither, mapped	50	150	100.0	25.0 [19.2, 31.6]	0/115 (4.8×10^{-35}) / 0/94 (1×10^{-28})	0
	Slither, strict	50	42	100.0	54.3 [43.6, 64.8]	0/7 (0.016) / 14/0 (1.2×10^{-4})	0
	EVMLiSA	37	71	74.0	34.3 [25.4, 44.0]	20/56 (4.4×10^{-5}) / 34/49 (0.12)	111
Labelled (81/389)	EVM-Guard v5	79	25	97.5	76.0 [66.6, 83.8]	–	0
	EVM-Guard v6	79	35	97.5	69.3 [60.0, 77.6]	–	0
	Slither, mapped	79	111	97.5	41.6 [34.5, 48.9]	1/87 (5.8×10^{-25}) / 0/76 (2.6×10^{-23})	14
	Slither, strict	79	29	97.5	73.1 [63.8, 81.2]	5/9 (0.42) / 10/4 (0.18)	14
	EVMLiSA	68	53	84.0	56.2 [46.9, 65.2]	12/40 (1.3×10^{-4}) / 20/38 (0.025)	90
Synthetic, Protocol 6 (30/210)	EVM-Guard v5	30	0	100.0	100.0 [88.4, 100.0]	–	0
	Slither, mapped	30	60	100.0	33.3 [23.7, 44.1]	0/60 (1.7×10^{-18})	0
	Slither, strict	30	30	100.0	50.0 [36.8, 63.2]	0/30 (1.9×10^{-9})	0
	EVMLiSA	30	60	100.0	33.3 [23.7, 44.1]	0/60 (1.7×10^{-18})	0
	eThor	30	87	100.0	25.6 [18.0, 34.5]	0/87 (1.3×10^{-26})	7

positives; it also failed to finish 27 SmartBugs files (21 timeouts and 6 crashes). EVMLiSA timed out on 111 SolidiFI files and 76 labelled files at 60 s, and 14 labelled files ended in error (including 13 whose sources do not compile and thus have no bytecode), giving 90 unfinished labelled files and lowering its recall on those corpora. With a limit of 600 s, and with a build that omits the CFG dump (which crashes on some graphs after the analysis has finished, and whose omission left the findings unchanged on 20 files that the standard build completes), EVMLiSA finished all SmartBugs-curated files, with 30 true and 28 false positives, and 55 of the 58 SolidiFI files that were rerun. Its false positives remained significantly more numerous than those of v5 ($p = 9.5 \times 10^{-7}$ and 2.2×10^{-12}). The static stage of Sailfish completed 138 SmartBugs files with 24 true and 4 false positives (85.7% precision, 77.4% recall). Its false positives do not differ significantly from those of v5 (4 negatives flagged by v5 alone and 1 by Sailfish alone, $p = 0.38$); it misses 7 positives that v5 finds and finds 2 that v5 misses. On the other corpora, however, it stopped with an internal assertion ("ORed count does not match") in its value-range analysis on all 50 SolidiFI reentrancy files, on 51 of the 81 labelled reentrancy files, and on all 30 injected-reentrancy and 30 mutex-guarded synthetic contracts. Scores on those corpora would therefore measure the failure rather than detection, and they are not reported.

Table 15 Protocol 7: frozen engines v5 and v6 on the same files, reentrancy. Precision with exact 95% Clopper–Pearson intervals (recall and FPR intervals are in results/protocol7/scores.json). The last two columns give the exact McNemar test of v5 against v6 on the negatives (FP) and on the positives (TP): files flagged by v5 only / by v6 only, and the two-sided p-value. Neither engine had been run on these corpora before the freeze, except v5 on SolidiFI and the labelled benchmark, whose v5 figures are post hoc.

Corpus (positives)	Engine	TP	FP	FN	Recall	Precision [CI]	FPR	FP: v5/v6 only (p)	TP: v5/v6 only (p)
SolidiFI (50)	v5	50	35	0	100.0	58.8 [47.6, 69.4]	10.0	0/21 (9.5×10^{-7})	0/0 (1)
	v6	50	56	0	100.0	47.2 [37.4, 57.1]	16.0		
Labelled (81)	v5	79	25	2	97.5	76.0 [66.6, 83.8]	8.1	3/13 (0.0213)	1/1 (1)
	v6	79	35	2	97.5	69.3 [60.0, 77.6]	11.4		
injected05d, 0.5.12 (30)	v5	30	0	0	100.0	100.0 [88.4, 100.0]	0.0	0/0 (1)	0/0 (1)
	v6	30	0	0	100.0	100.0 [88.4, 100.0]	0.0		
injected8c, 0.8.20 (30)	v5	0	0	30	0.0	n/a	0.0	0/0 (1)	0/30 (1.9×10^{-9})
	v6	30	0	0	100.0	100.0 [88.4, 100.0]	0.0		

7 Engine v6 under a Pre-Registered Protocol

Protocol.

Engine v6 was developed on SmartBugs-curated, on five fixtures in tests/fixtures/ and on the spent corpus injected8b, so its figures on these sets are post hoc. Protocol 7 (docs/ENGINE_V6_EVAL_PROTOCOL.md) was then frozen; SHA-256 digests of the protocol, the labelling rubric, the corpus builders and the engine manifest are recorded in docs/ENGINE_V6_EVAL_FREEZE.sha256, and byte copies of the v6 sources are kept in archive/engine_v6/. The protocol specifies five corpora on which v6 had never been run: SolidiFI, the labelled benchmark, two new synthetic corpora built by the earlier generators with new seeds (injected05d, solc 0.5.12, seed 20261101; injected8c, solc 0.8.20, seed 20261102; 210 files each), and the deployed contracts described in Section 8. Each corpus was analysed once with the frozen v5 and v6 engines, and the protocol requires the results to be reported whatever they are. Two files outside the engine were changed after the freeze, in both cases before any engine had been run on the corpus concerned; the changes are recorded in docs/ENGINE_V6_EVAL_ADDENDUM.md. The engine sources matched the manifest when the runs began.

Results.

The reentrancy results are shown in Table 15. On the two synthetic corpora, both engines are exact on Solidity 0.5.12, and on Solidity 0.8.20 v6 finds all 30 injected reentrancies without a false positive, whereas v5 finds none ($p = 1.9 \times 10^{-9}$). It is return-site tracking that makes 0.8 bytecode analysable. On the two real benchmarks, v6 retains the recall of v5 but adds false positives: 21 on SolidiFI ($p = 9.5 \times 10^{-7}$) and a net 10 on the labelled benchmark (13 added and 3 removed, $p = 0.021$). All 21 additional SolidiFI findings are classified as *same-function, feasible*, and 7 of these files also carry an *unknown* finding. The additional candidates therefore come from code that return-site tracking makes reachable, rather than from candidates that the solver failed to decide. Z3 confirms that a path to the call and a re-entrant run exist, but the attacker model cannot establish that re-entry would benefit the attacker; the callee

may be, for example, a seller who would only pay itself. On the labelled benchmark, v6 also adds 6 arithmetic false positives ($p = 0.031$), which arise from the same newly reachable code. On injected8c, it flags all 30 denial-of-service positives together with all 30 bounded-loop negatives, which shows that its loop check does not carry over to 0.8 bytecode; no claim is made for denial of service.

Consequence.

Engine v6 is thus the better choice for Solidity 0.8 bytecode, and it records the kind of each reentrancy, but on Solidity 0.4–0.5 code it is significantly less precise than v5. For this reason, v5 is retained as the default engine, and the v5 figures elsewhere in this paper are unchanged. On SmartBugs-curated (post hoc), v6 reaches 29 true positives with 4 false positives (87.9%). Compared with v5, it removes the three privileged-caller false positives and both copies of PoCGame discussed in Section 5.3, finds reentrancy_bonus, misses TokenBank, whose withdrawal function is restricted to the owner through a shadowed owner variable, and adds two unknown findings on Parity wallet files.

8 Deployed Contracts

Corpus.

The corpus was defined in the protocol before any engine was run on it. It consists of every entry of the smart-contract-sanctuary mainnet index (commit 015d0105, index SHA-256 fdb990d8…) compiled with Solidity 0.4 or 0.5, excluding the 37,892 addresses that appear anywhere in the earlier corpora, that held a non-zero ether balance and had code at a single block (src/eval/build_mainnet_catalog.py). Of the 43,998 candidates, 3,214 contracts held ether at block 26,319,084, 45,217 ether in total (median 0.011 ether; 413 contracts held at least 1 ether). The engines analysed the runtime code read from the chain, not a recompilation.

Scan.

Table 16 summarises the run. Engine v5 flagged 522 contracts (16.2%) and v6 978 (30.4%). Of the 978 contracts flagged by v6, 578 reached a search limit and were kept as unknown, which indicates that on real code the solver rejects too few candidates to offset the additional code exposed by return-site tracking. Neither replay tier confirmed any finding. A likely explanation is that the default-state replay starts from empty storage, in which deployed contracts hold no balances to withdraw, while the witness replay constrains the starting storage but not the external contracts on which real code depends. Return-site tracking allowed v6 to resolve every jump in 1,568 more contracts than v5.

Hand check.

As specified in the protocol, a uniform random sample of 120 contracts was drawn from the 1,070 contracts flagged by either engine (random.Random(7) over the

Table 16 Frozen engines on 3,214 deployed Solidity 0.4–0.5 contracts holding ether. Analysis time is per contract and excludes replay; wall time per contract includes replay. Neither engine reported an error.

	v5	**v6**
Contracts flagged for reentrancy	522	978
flagged by both engines	430	
Ether held by flagged contracts	31,290	42,057
Flagged contracts holding ≥ 1 ether	78	183
Analysis time, median / mean (s)	0.48 / 0.46	0.89 / 2.70
Analysis time, 95th percentile / max (s)	1.0 / 5.2	12.5 / 48.1
Wall time per contract with replay (s)	1.3	4.0
Contracts with an unresolved jump	2,816	1,248
Confirmed by default-state replay	0	0
Confirmed by witness replay	n/a	0
v6 reentrancy findings by kind and feasibility verdict		
same-function, feasible		385
cross-function, feasible		6
read-only, feasible		9
unclassified, search limit reached		578

sorted addresses). Verified sources were obtained from the sanctuary at the pinned commit (76 contracts) or, for addresses that its index lists without a file, from Sourcify, where the verified runtime bytecode matches the deployed code, in some cases apart from the metadata hash (44 contracts). The sources were exported with comments and contract names removed and in shuffled order (src/eval/blind_export.py), and each contract was labelled for reentrancy according to the rubric in docs/LABELLING_RUBRIC.md, without knowledge of the engine that had flagged it. Under this rubric, a contract is *exploitable* only if a call forwards more than the stipend to an address that the attacker controls, a later write or a stale read exists, an unprivileged account can take the path, and re-entry yields a gain. The labels were locked with a SHA-256 digest before the key was opened. One contract was labelled exploitable and 119 not exploitable; none was labelled privileged-only or undetermined. For comparison, Slither 0.11.6 was run on the verified sources of the same 120 contracts.

Results.

Table 17 gives the precision on the sample. All four configurations flag the one exploitable contract, and precision on deployed code is low for every tool. Engine v5 reached 1.6%, with fewer false positives than strict Slither on the same contracts ($p = 0.021$) and considerably fewer than v6. The 119 false positives fall into a small number of groups. In 50 contracts the callee is fixed, being a constant, an address set in the constructor or a contract configured by an administrator, often a token or dividend contract of the same project. Another 21 are multisignature wallets that mark a transaction as executed before the call and accept only their owners. In 19, every

Table 17 Blind hand check of 120 deployed contracts sampled from those flagged by v5 or v6. Precision counts only contracts labelled exploitable as correct, with exact 95% Clopper–Pearson intervals. The sample is drawn from EVM-Guard's flags, so the Slither figures are conditional on that selection. McNemar: false positives of v5 only / of the other tool only.

Tool	Flagged	Exploitable	Precision [CI]	vs v5 FP (p)
EVM-Guard v5	62	1	1.6 [0.04, 8.7]	–
EVM-Guard v6	107	1	0.9 [0.02, 5.1]	13/58 (6.3×10^{-8})
Slither, strict	79	1	1.3 [0.03, 6.9]	16/33 (0.021)
Slither, mapped	114	1	0.9 [0.02, 4.8]	1/53 (6.1×10^{-15})

ether transfer uses the stipend and the remaining calls go to fixed contracts such as the Oraclize resolver, and in 14 the state is settled before the call. Ten can be reached only by a privileged caller; in 4 the callee is supplied by the attacker, but re-entry affects only the attacker's own entries; and in one the function always reverts. Most of these cases involve an address that cannot be classified from bytecode alone, which appears to be the main limitation of a bytecode-only attacker model on real code. Precision on deployed contracts is thus far below the benchmark figures, and the output of EVM-Guard on such code should be treated as a list for manual review. An independent second rater, without access to the first set of labels, labelled 15 of the 120 contracts, including the exploitable one, and agreed with the first rater on all 15 (Cohen's $\kappa = 1.0$; Section 10.1).

Disclosure.

The exploitable contract, compiled with solc 0.4.24, manages a payout queue: its public function $\mathrm{payout}()$ forwards 1,000,000 gas to each depositor and advances the queue index only after the call, so that a re-entering depositor can cause the loop to skip the rest of the queue. At the corpus block, the contract held less than 0.001 ether. No transaction was executed against it. Its source names no author, project or reporting channel, so no report could be sent, and, in accordance with the protocol, its address is not published ($\mathrm{docs/DISCLOSURE_LOG.md}$).

9 Discussion

Where EVM-Guard is competitive.

On the Protocol 6 synthetic corpus, v5 reached 100% reentrancy recall and precision, compared with 26–50% precision for Slither, Mythril, EVMLiSA and eThor on the same files. On the synthetic corpora of Protocol 7, both engines were exact on Solidity 0.5.12, and v6 was also exact on Solidity 0.8.20. On the real benchmarks, v5 produced significantly fewer reentrancy false positives than EVMLiSA and eThor on every corpus on which they were run, and fewer than Slither when all of its reentrancy detectors are counted. On deployed contracts, it produced fewer false positives than strict Slither on the same sample. Since it requires only bytecode, it can also be applied where no source code is available.

Where it is not.

Against Slither restricted to its two exploitable-reentrancy detectors, the advantage of v5 is reduced to a significant difference on SolidiFI and no significant difference on SmartBugs-curated and the labelled benchmark (Section 6). On deployed code, the precision of every configuration was below 2% (Section 8). Most flagged calls there go to fixed or trusted addresses, which a bytecode-only attacker model cannot classify, and replay from empty storage confirmed none of them. The results of engine v6 suggest that feasibility checking alone does not remove such false positives: return-site tracking exposes more code, and the solver can reject a path only if it is infeasible, not if it is feasible but harmless. No claim is made for denial of service or arithmetic (Sections 5.3 and 5.6).

Real benchmarks and synthetic tests.

Engine v5 was developed on SmartBugs-curated, SolidiFI and the labelled benchmark, so its figures on these corpora (80.6%, 58.8% and 76.0%) are post hoc, whereas the v6 figures on SolidiFI and the labelled benchmark are not. The synthetic tests rely on template injection, so their precision cannot be compared with the figures on real benchmarks. The sample of deployed contracts therefore provides the only estimate of precision on real code that was not used during development.

Research questions.

With respect to the first research question, CFG dataflow with the stipend bound and the mutex rules produced fewer reentrancy false positives than the bytecode analysers and than Slither under the broad mapping, at equal or higher recall; it did not produce significantly fewer than strict Slither on two of the three real benchmarks, and the figures on those benchmarks are post hoc. With respect to the second, default-state replay confirmed 8 findings on SmartBugs-curated, one of which is a false alarm, at a considerable cost in recall, and neither replay tier confirmed any finding on deployed contracts (Sections 5.4 and 8). With respect to the third, precision on the synthetic tests (100% for v5 and v6) cannot be compared with precision on the real benchmarks, because the distributions differ. Finally, with respect to the fourth, v5 flagged 16.2% of 3,214 deployed contracts at a median of 0.48 s per contract, and 1 of the 62 sampled contracts flagged by v5 proved exploitable.

Specification and implementation.

The specification of Section 3 describes what a path-complete analysis would decide. Engine v5 approximates it on the CFG without solving path conditions, while engine v6 solves them for bounded paths under the model of Remark 1; a search that reaches a limit decides nothing.

10 Threats to Validity

10.1 Construct Validity

SmartBugs-curated assigns one category to each file, and a file is treated as a negative for every other category. A file that contains an unlabelled instance of category X is therefore counted as a false positive for any tool that flags it, which may understate precision and overstate false-positive rates for all tools. The arithmetic category concerns overflow, not the truncation property that EVM-Guard checks. On the labelled benchmark, 58 of the 92 reentrancy false positives of v2, and 5 of the 25 of v5, lie in a folder labelled TOD, which the protocol scores as negative; since these flags were not reviewed by hand, they may include unrecorded reentrancy. The access-control mapping of EVM-Guard uses only SWC-112 and SWC-124, which cover a small part of what SmartBugs labels as access control. The labels of the deployed contracts were assigned by one rater from verified source code, without knowledge of the engine and according to a written rubric. To assess the reliability of these labels, an independent second rater labelled 15 of the 120 contracts from the same blind export, using the rubric in the form of four yes/no questions together with excerpts of the functions that make external calls and the full source. The subset consisted of the exploitable contract and 14 others drawn at random, presented in shuffled order, and the two raters agreed on all 15 contracts (Cohen's $\kappa = 1.0$, results/protocol7/rater_agreement.json). Since the subset is small and contains a single positive, this agreement indicates that the labels are reproducible for clear cases, but it cannot rule out errors among the remaining 105 contracts, which were labelled by one rater only. The rubric also requires a gain from re-entry, which is stricter than Definition 2, so that precision on this sample refers to exploitable reentrancy rather than to the pattern.

10.2 Internal Validity

Each protocol was fixed before the run that it governs, but the engine was revised after the results on SmartBugs-curated, SolidiFI and the labelled benchmark were known (v2, v3 and v5). The pre-registered runs are the v1 run on SmartBugs-curated, the v2 runs on SolidiFI and the labelled benchmark, the synthetic tests of Sections 5.9 and 5.10, the Protocol 5 run, and all runs of Protocol 7. The baseline mappings (Addendum A of docs/EXTERNAL_EVAL_PROTOCOL.md) were written after the first EVM-Guard run and before any baseline was run. After the v5 runs, four engine source files were changed, and byte copies of the sources that produced those runs were not kept; they could not be recovered from version control. The v5 figures therefore rest on reproduction: the current sources with the v5 configuration (docs/V5_REPRODUCTION_MANIFEST.sha256) reproduce the v5 flags and replay confirmations of every file on all five corpora (docs/REPRODUCIBILITY.md). To avoid this problem for v6, its sources were archived byte for byte at the freeze (archive/engine_v6/), and the v5 runs of Protocol 7 use the same sources with the v5 configuration. Each baseline was run once per file, and the timeouts and errors of Mythril (37 and 20 on SmartBugs-curated) are

counted as unflagged, which lowers its recall. Finally, the CFG resolves only constant jump targets; 180 JUMP blocks in SmartBugs-curated have no resolved successor, which may hide paths and cause false negatives.

10.3 External Validity

The contracts of SmartBugs-curated are small (mean 63.4 source lines) and compile with solc 0.4.0–0.5.0, versions that predate checked arithmetic and EIP-1153. The results may therefore not apply to modern Solidity, to large production protocols, or to compilers that emit different code layouts, on which the checks of EVM-Guard depend. On an earlier corpus of 210 contracts compiled with solc 0.8.20, 60 of which contain injected reentrancy or denial-of-service patterns (docs/INJECTED_EVAL_PROTOCOL.md), EVM-Guard produced no finding in either category. Two causes were identified. First, an earlier disassembler decoded PUSH0 as INVALID, which stopped CFG construction at the dispatcher; this has since been corrected. Second, the abstract stack merges the return addresses of the shared internal helpers that 0.8 compilers emit for checked arithmetic, which cuts the dataflow within a function. Engine v6 removes the second cause with return sites tracked per context and detects all 30 injected reentrancies of a fresh 0.8.20 corpus (Section 7). That corpus is synthetic, however, the loop check of v6 does not carry over to 0.8, and v6 is not the default engine. The claims of this paper are therefore limited to Solidity 0.4–0.5 bytecode, the compiler range that also applies to SolidiFI (all files compile with solc 0.5.12).

10.4 Statistical Conclusion Validity

Several categories have few positives (6 for denial of service, 15 for arithmetic and 18 for access control), and their Clopper–Pearson intervals are wide. Comparisons between tools are based on exact McNemar tests on the same files, which take into account the pairing that marginal intervals ignore. Many such tests are reported without correction for multiplicity. The reentrancy comparisons on which the conclusions rest have $p < 10^{-4}$, with the exception of those against strict Slither and those on the labelled benchmark, which are discussed individually. The sample of deployed contracts contains a single exploitable contract, so its precision intervals are wide, and the comparison between tools on that sample rests on false positives alone.

11 Conclusion

This paper has described EVM-Guard, a bytecode-only triage tool for reentrancy in Solidity 0.4–0.5 contracts. Engine v5 flags an external call that is followed by a storage write without a mutex in a CFG with an abstract-stack dataflow, excludes calls that forward only the stipend, and replays candidates in a sandbox. On SmartBugs-curated, it reached 93.5% recall at 80.6% precision (post hoc), with significantly fewer false positives than EVMLiSA, eThor and Slither with its full set of reentrancy detectors, and no significant difference from Slither restricted to its two exploitable-reentrancy

detectors. Engine v6 adds return-site tracking and Z3 path feasibility. Under a pre-registered protocol, it extended detection to Solidity 0.8 bytecode but produced more false positives on Solidity 0.5 benchmarks, and v5 has therefore been retained as the default engine. On 3,214 deployed contracts holding ether, v5 required less than a second per contract, and a blind review of a sample of flagged contracts identified one exploitable contract, a precision of 1.6%. These findings suggest that the main obstacle on deployed code is to decide whether a callee is trusted, which bytecode alone rarely reveals. Further work is therefore needed to model the on-chain state at the call, including the configured addresses and the code deployed at them.

Declarations

- **Funding:** This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.
- **Conflict of Interest / Competing Interests:** The authors declare no competing financial or non-financial interests related to this work.

- **Data Availability:** The pre-registered protocols, catalogues, SolidiFI and synthetic sources, and the outputs of every reported run are available at https://github.com/RaffeOP/EVM-Guard. SmartBugs-curated (commit 230e6491) and the labelled benchmark (commit 8b903aa) are not redistributed; the script reproduce.sh clones them at these commits. The corpus of deployed contracts can be rebuilt from a public index and a chain node with src/eval/build_mainnet_catalog.py. The labels and scores of the manual review are released in aggregate form, and the key that links the sample to addresses is withheld because one sampled contract is exploitable (docs/DISCLOSURE_LOG.md).
- **Code Availability:** The EVM-Guard analyser (engines v5 and v6), the Z3 encoders, the replay oracles and the evaluation scripts are available at https://github.com/RaffeOP/EVM-Guard under the MIT licence. The script reproduce.sh rebuilds every corpus with the pinned compilers and package versions and reruns engine v5, and docs/REPRODUCIBILITY.md lists the expected results, the differences from the frozen engine manifest, and the commands for the v6, baseline and deployed-contract runs.
- **Authors' Contributions:** Devarsh Patel conceptualized the study, developed the EVM-Guard analysis engine and replay oracles, conducted the empirical evaluations, and drafted the manuscript. Chandru Vignesh C supervised the research, contributed to the analytical framing, reviewed and verified the empirical findings, and critically reviewed and edited the manuscript. Both authors read and approved the final manuscript.
- **Use of Artificial Intelligence:** During the preparation of this manuscript, the authors utilized large language model (LLM) tools solely to assist with manuscript typesetting, code formatting, style consistency, and language polishing. The tools were not used to generate research concepts, formal specifications, mathematical formulations, experimental designs, empirical analyses, or scientific conclusions. All core contributions, implementations, evaluation protocols, and analyses were

independently conceived, executed, and verified by the authors, who assume full responsibility for the entire content and integrity of this work.

References

[1] Zhou, L., Xiong, X., Ernstberger, J., Chaliasos, S., Wang, Z., Wang, Y., Qin, K., Wattenhofer, R., Song, D., Gervais, A.: SoK: Decentralized finance (DeFi) attacks. In: 2023 IEEE Symposium on Security and Privacy (SP), pp. 2444–2461. IEEE, San Francisco, CA, USA (2023). https://doi.org/10.1109/SP46215.2023.10179435

[2] Chen, J., Xia, X., Lo, D., Grundy, J., Luo, X., Chen, T.: Defining smart contract defects on Ethereum. IEEE Transactions on Software Engineering **48**(1), 327–345 (2022). https://doi.org/10.1109/TSE.2020.2989002

[3] Daian, P., Goldfeder, S., Kell, T., Li, M., Zhao, X., Bentov, I., Breidenbach, L., Juels, A.: Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 910–927. IEEE, San Francisco, CA, USA (2020). https://doi.org/10.1109/SP40000.2020.00040

[4] Feist, J., Grieco, G., Groce, A.: Slither: A static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), pp. 8–15. IEEE, Montreal, QC, Canada (2019). https://doi.org/10.1109/WETSEB.2019.00008

[5] Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Bünzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 67–82. ACM, Toronto, Canada (2018). https://doi.org/10.1145/3243734.3243780

[6] Jordan, H., Scholz, B., Subotić, P.: Soufflé: On synthesis of program analyzers. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, pp. 422–430. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-41540-6_23

[7] Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE), pp. 530–541. ACM, Seoul, South Korea (2020). https://doi.org/10.1145/3377811.3380364

[8] Ghaleb, A., Pattabiraman, K.: How effective are smart contract analysis tools? Evaluating smart contract static analysis tools using bug injection. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software

Testing and Analysis (ISSTA), pp. 415–427. ACM, Virtual Event, USA (2020). https://doi.org/10.1145/3395363.3397385

[9] Mossberg, M., Manzano, F., Hennenfent, E., Groce, A., Grieco, G., Feist, J., Brunson, T., Dinaburg, A.: Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1186–1189. IEEE, San Diego, CA, USA (2019). https://doi.org/10.1109/ASE.2019.00133

[10] Luu, L., Chu, D.-H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 254–269. ACM, Vienna, Austria (2016). https://doi.org/10.1145/2976749.2978309

[11] de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, pp. 337–340. Springer, Berlin, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78800-3_24

[12] Krupp, J., Rossow, C.: teEther: Gnawing at Ethereum to automatically exploit smart contracts. In: 27th USENIX Security Symposium (USENIX Security 18), pp. 1317–1333. USENIX Association, Baltimore, MD, USA (2018). https://www.usenix.org/conference/usenixsecurity18/presentation/krupp

[13] Grieco, G., Song, W., Cygan, A., Feist, J., Groce, A.: Echidna: Effective, usable, and fast fuzzing for smart contracts. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 557–560. ACM, Virtual Event, USA (2020). https://doi.org/10.1145/3395363.3404366

[14] Torres, C.F., Iannillo, A.K., Gervais, A., State, R.: ConFuzzius: A data dependency-aware hybrid fuzzer for smart contracts. In: 2021 IEEE European Symposium on Security and Privacy (EuroS&P), pp. 103–119. IEEE, Vienna, Austria (2021). https://doi.org/10.1109/EuroSP51992.2021.00018

[15] Dxo, Soos, M., Paraskevopoulou, Z., Lundfall, M., Brockman, M.: hevm, a fast symbolic execution framework for EVM bytecode. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, pp. 453–465. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-65627-9_22

[16] Permenev, A., Dimitrov, D., Tsankov, P., Drachsler-Cohen, D., Vechev, M.: VerX: Safety verification of smart contracts. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 1661–1677. IEEE, San Francisco, CA, USA (2020). https://doi.org/10.1109/SP40000.2020.00024

[17] Hildenbrandt, E., Saxena, M., Rodrigues, N., Zhu, X., Daian, P., Guth, D.,

Moore, B., Park, D., Zhang, Y., Stefanescu, A., Rosu, G.: KEVM: A complete formal semantics of the Ethereum virtual machine. In: 2018 IEEE 31st Computer Security Foundations Symposium (CSF), pp. 204–217. IEEE, Oxford, UK (2018). https://doi.org/10.1109/CSF.2018.00022

[18] Grech, N., Brent, L., Scholz, B., Smaragdakis, Y.: Gigahorse: Thorough, declarative decompilation of smart contracts. In: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), pp. 1176–1186. IEEE, Montreal, QC, Canada (2019). https://doi.org/10.1109/ICSE.2019.00120

[19] Schneidewind, C., Grishchenko, I., Scherer, M., Maffei, M.: eThor: Practical and provably sound static analysis of Ethereum smart contracts. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 621–640. ACM, Virtual Event, USA (2020). https://doi.org/10.1145/3372297.3417250

[20] Arceri, V., Merenda, S.M., Negrini, L., Olivieri, L., Zaffanella, E.: EVMLiSA: Sound static control-flow graph construction for EVM bytecode. Blockchain: Research and Applications **7**(4), 100384 (2026). https://doi.org/10.1016/j.bcra.2025.100384

[21] Bose, P., Das, D., Chen, Y., Feng, Y., Kruegel, C., Vigna, G.: SAILFISH: Vetting smart contract state-inconsistency bugs in seconds. In: 2022 IEEE Symposium on Security and Privacy (SP), pp. 161–178. IEEE, San Francisco, CA, USA (2022). https://doi.org/10.1109/SP46214.2022.9833721

[22] Rodler, M., Li, W., Karame, G.O., Davi, L.: Sereum: Protecting existing smart contracts against re-entrancy attacks. In: Proceedings 2019 Network and Distributed System Security Symposium (NDSS). Internet Society, San Diego, CA, USA (2019). https://doi.org/10.14722/ndss.2019.23413

[23] Grossman, S., Abraham, I., Golan-Gueta, G., Michalevsky, Y., Rinetzky, N., Sagiv, M., Zohar, Y.: Online detection of effectively callback free objects with applications to smart contracts. Proceedings of the ACM on Programming Languages **2**(POPL), 1–28 (2017). https://doi.org/10.1145/3158136

[24] Grech, N., Kong, M., Jurisevic, A., Brent, L., Scholz, B., Smaragdakis, Y.: MadMax: Surviving out-of-gas conditions in Ethereum smart contracts. Proceedings of the ACM on Programming Languages **2**(OOPSLA), 1–27 (2018). https://doi.org/10.1145/3276486

[25] Grishchenko, I., Maffei, M., Schneidewind, C.: A semantic framework for the security analysis of Ethereum smart contracts. In: Principles of Security and Trust (POST). Lecture Notes in Computer Science, pp. 243–269. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-89722-6_10

[26] Albert, E., Correas, J., Gordillo, P., Román-Díez, G., Rubio, A.: GASOL: Gas analysis and optimization for Ethereum smart contracts. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, pp. 118–125. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-45237-7_7

[27] Perez, D., Livshits, B.: Broken metre: Attacking resource metering in EVM. In: Proceedings 2020 Network and Distributed System Security Symposium (NDSS). Internet Society, San Diego, CA, USA (2020). https://doi.org/10.14722/ndss.2020.24267

[28] Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL), pp. 238–252. ACM, Los Angeles, CA, USA (1977). https://doi.org/10.1145/512950.512973

[29] Ferreira, J.F., Cruz, P., Durieux, T., Abreu, R.: SmartBugs: A framework to analyze Solidity smart contracts. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1349–1352. ACM, Virtual Event, Australia (2020). https://doi.org/10.1145/3324884.3415298

[30] Clopper, C.J., Pearson, E.S.: The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika **26**(4), 404–413 (1934). https://doi.org/10.1093/biomet/26.4.404

[31] McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika **12**(2), 153–157 (1947). https://doi.org/10.1007/BF02295996